

pagmaxx



Documentação da API

Guia de integração do PagMaxx Gateway: vendas com cartão, Pix, links de pagamento, tokenização, autenticação 3DS, assinaturas e webhooks.

Sumário

COMEÇANDO

Documentação da API PagMaxx	4
Autenticação	7
API Keys (server-to-server)	11
Erros & Status codes	14

VENDAS COM CARTÃO

Criar uma venda	18
Pagamento com 3DS obrigatório	28
Cancelar transação	34

LINKS DE PAGAMENTO

Criar Link de Pagamento	39
Consultar Links de Pagamento	44
Atualizar Link de Pagamento	50
Criar Link com Checkout PagMaxx (com split)	54

PIX

Venda Pix	61
Obter Venda Pix	66

CARTÃO TOKENIZADO E 3DS

Tokenizar o Cartão	69
3DS Autenticação — Setup	73
3DS Autenticação	78
3DS Challenge Result	89

TAXAS

Simular Taxa	92
--------------------	----

RECORRÊNCIA

INTEGRAÇÃO

Webhooks 119

Documentos de Credenciamento 127

Documentação da API PagMaxx

Bem-vindo à documentação oficial da PagMaxx Gateway. Aqui você integra pagamentos com cartão de crédito, Pix, links de pagamento, tokenização de cartões, autenticação 3D Secure, assinaturas (recorrência) e simulação de taxas — tudo sobre uma infraestrutura segura e compatível com PCI-DSS.

Visão geral

Toda a API é REST, usa JSON em requisições e respostas e é servida sob o prefixo `/api` em HTTPS. Cada estabelecimento (EC) opera com suas próprias credenciais e só enxerga as próprias transações.

Os endpoints de pagamento processam a transação na adquirente e repassam o resultado, acrescentando o objeto `_pagmaxx` com os identificadores internos (ex.: `payment_id`). O HTTP status reflete o resultado real na adquirente.

Ambientes

Use Homologação para testar com cartões de teste e Produção para transações reais.

Base URLs

Homologação	<code>https://api.homolog.pagmaxx.com/api</code>
Produção	<code>https://api.prod.pagmaxx.com/api</code>

 Copiar

Dica

Em cada página desta documentação há um seletor **Homologação / Produção** e um testador interativo (cURL, Node.js e Python) que monta a requisição para você.

Autenticação

A API aceita dois métodos de autenticação:

- **Bearer JWT** — gere um `access_token` em `POST /auth/token` e envie no header `Authorization`. Usado por todos os endpoints. Veja [Autenticação](#).
- **API Key (server-to-server)** — envie o header `X-API-Key`. Aceito nos endpoints de integração de pagamento (pay-secure, tokenização e 3DS). Veja [API Keys](#).

Início rápido (do zero ao primeiro pagamento)

1. Obtenha um Access Token

```
curl -X POST https://api.prod.pagmaxx.com/api/auth/token \
-H "Content-Type: application/json" \
-d '{
  "email": "voce@empresa.com",
  "password": "sua-senha"
}'
```

[Copiar](#)

Resposta

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR...",
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR...",
  "token_type": "bearer"
}
```

[Copiar](#)

2. Crie uma venda com cartão

```
curl -X POST https://api.prod.pagmaxx.com/api/payments/pay \
-H "Authorization: Bearer <ACCESS_TOKEN>" \
-H "Content-Type: application/json" \
-d '{
  "amount": 100.50,
  "currency": "BRL",
  "type": "CREDIT",
  "cardNumber": "4111111111111111",
  "cardSecurityCode": "123",
  "cardExpirationDate": "1229",
  "cardHolderName": "JOAO SILVA",
  "installments": 1
}'
```

[Copiar](#)

Integração server-to-server

Em integrações backend, prefira POST /payments/pay-secure com o header X-API-Key. Essa rota também exige 3DS aprovado, reduzindo chargeback e transferindo a responsabilidade da fraude ao emissor.

Serviços disponíveis

- **Cartão de crédito** — venda (/payments/pay), venda com 3DS obrigatório (/payments/pay-secure) e cancelamento/estorno (/payments/void).
- **3D Secure 2.x** — setup, autenticação e challenge result.
- **Pix** — criação de cobrança Pix e consulta de status.

- **Links de pagamento** — criar, consultar e atualizar.
- **Tokenização** — armazenamento seguro de cartões (slugToken / cofre).
- **Assinaturas** — recorrência no cartão (MIT) e Pix.
- **Simulador de taxas** — taxas e valor líquido por bandeira/parcelamento.

Segurança

Todas as requisições trafegam por HTTPS. A plataforma segue padrões internacionais:

- PCI-DSS para o manuseio de dados de cartão
- 3D Secure 2.x (autenticação do portador)
- Tokenização de dados sensíveis (PAN nunca trafega no seu backend quando você usa slugToken)
- Tokens de curta expiração e API Keys com escopo restrito a pagamento

[Autenticação](#) →

Autenticação

Produção

POST /api/auth/token

Autentique com e-mail e senha para receber um **Access Token** (válido por tempo curto) e um **Refresh Token**. O Access Token é enviado no header Authorization: Bearer <token> em todas as chamadas da API.

JWT ou API Key?

O fluxo de e-mail/senha (JWT) é ideal para o seu painel/backoffice. Para integração **server-to-server** sem armazenar senhas, use uma [API Key](#) (header X-API-Key).

Exemplo de corpo

```
{
  "email": "voce@empresa.com",
  "password": "sua-senha"
}
```

 Copiar

Parâmetros do Body

email email **required**

E-mail da conta no portal PagMaxx.

password password **required**

Senha da conta.

Testar requisição

cURL

Node.js

Python

```
curl -X POST "https://api.prod.pagmaxx.com/api/auth/token" \  
\  
-H "Content-Type: application/json" \  
-d '{}'
```

[Copiar](#)

Resposta:

● ---

```
// Aguardando requisição...
```

Parâmetros do corpo

email	E-mail da conta.
-------	------------------

string **obrigatório**

password	Senha da conta.
----------	-----------------

string **obrigatório**

Resposta

● HTTP 200

```
{  
  "access_token": "eyJhbGciOiJIUzI1NiIsInR...",  
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR...",  
  "token_type": "bearer"  
}
```

[Copiar](#)

Use o access_token no header das requisições: Authorization: Bearer <access_token>.

Renovar o token — POST /api/auth/refresh

Quando o Access Token expirar, troque o refresh_token por um novo par de tokens, sem pedir a senha novamente.

```
curl -X POST https://api.prod.pagmaxx.com/api/auth/refresh \
-H "Content-Type: application/json" \
-d '{ "refresh_token": "<REFRESH_TOKEN>" }'
```

[Copiar](#)

● HTTP 200

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR...\"",
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR...\"",
  "token_type": "bearer"
}
```

[Copiar](#)

Limites e erros

⚠ Rate limit

/auth/token: 5 requisições/minuto · /auth/refresh: 20 requisições/minuto. Ao exceder, a API responde 429.

STATUS	DETAIL	QUANDO OCORRE
400	Email ou senha incorretos	Credenciais inválidas em /auth/token.
403	Conta pendente de aprovação. Nossa equipe está analisando seu cadastro.	Conta ainda não aprovada.
401	Refresh token inválido ou expirado	Refresh token inválido/expirado em /auth/refresh.
429	Rate limit exceeded	Excedeu o limite de requisições por minuto.
500	Erro interno	Falha inesperada no servidor.

[← Introdução](#)[API Keys →](#)

API Keys (server-to-server)

Para integrações de backend, a PagMaxx oferece autenticação por **API Key** em vez de e-mail/senha. A chave é enviada no header X-API-Key e dispensa o fluxo de login/refresh — ideal para servidores que processam pagamentos automaticamente.

Como usar

Basta enviar o header X-API-Key com a sua chave. Quando presente, ele substitui o Authorization: Bearer.

```
curl -X POST https://api.prod.pagmaxx.com/api/payments/pay-secure \
-H "X-API-Key: pmx_live_9f3a2b1c4d5e6f7a8b9c0d1e2f3a4b5c" \
-H "Content-Type: application/json" \
-d '{
  "amount": 100.50,
  "currency": "BRL",
  "type": "CREDIT",
  "slugStoredCard": "9F3A2B1C4D5E6F7A8B9C0D1E2F3A4B5C",
  "cardSecurityCode": "123",
  "threeDsData": { "cavv": "...", "secureVersion": "2.2.0" }
}'
```

 Copiar

Endpoints que aceitam X-API-Key

Por segurança, a API Key tem **escopo restrito a pagamento**. Ela funciona nos endpoints de integração de pagamento; operações sensíveis (estorno, gestão de conta, usuários, split) continuam exigindo login no painel (JWT).

POST /payments/pay-secure Venda com cartão e 3DS obrigatório.

X-API-Key | JWT opcional

POST /payments/tokenize-card Tokenização de cartão.

X-API-Key | JWT opcional

POST
/payments/3ds/authentication-
setup Setup da autenticação 3DS.

X-API-Key | JWT opcional

POST
/payments/3ds/authentication Autenticação 3DS.

X-API-Key | JWT opcional

Demais endpoints

Endpoints como /payments/pay (legado), /payments/void, /payment-link/*, /payments/pix/*, /payments/simulate-fee e /subscriptions/* usam **Bearer JWT**.

Emissão e ciclo de vida

As chaves são emitidas pela equipe PagMaxx (backoffice) para o seu estabelecimento. No momento da criação, o segredo completo é exibido **uma única vez** — guarde-o com segurança. Depois disso só o `key_prefix` fica visível.

● Exemplo do retorno na criação (mostrado uma única vez)

```
{
  "id": "8c1d2e3f-...",
  "label": "Integração loja virtual",
  "key_prefix": "pmx_live_9f3a2b1c",
  "api_key": "pmx_live_9f3a2b1c4d5e6f7a8b9c0d1e2f3a4b5c",
  "pagmaxx_customer_id": "4f9c1d2e-...",
  "created_at": "2026-06-17T12:00:00Z"
}
```

 Copiar

- A chave é armazenada apenas como hash (SHA-256) — a PagMaxx não consegue recuperá-la.
- Pode ser **revogada** a qualquer momento; chamadas com chave revogada retornam 401.
- Cada chave registra o `last_used_at` para auditoria.

Limites e segurança



Rate limit por chave

Cada API Key tem um limite de requisições por minuto. Ao exceder, a API responde 429 – Muitas tentativas. Tente mais tarde.



Boas práticas

Use chaves diferentes por integração/ambiente, nunca exponha a chave no front-end ou em repositórios, e revogue imediatamente qualquer chave comprometida.

Erros de autenticação: 401 – API key invalida ou revogada. / 401 – Estabelecimento inativo.

[← Autenticação](#)

[Erros & Status →](#)

Erros & Status codes

A API usa os códigos de status HTTP convencionais para indicar o resultado de uma requisição. **2xx** = sucesso, **4xx** = problema na requisição (dados, autenticação ou regra de negócio) e **5xx** = falha no servidor ou na adquirente.

Formato do erro

Erros de negócio retornam um objeto com o campo `detail` em formato de texto:

```
{
  "detail": "Pagamento não encontrado"
}
```

[Copiar](#)

Erros de **validação do corpo** (campos obrigatórios ausentes, tipos ou formatos inválidos) retornam 422 com `detail` em formato de lista:

```
{
  "detail": [
    {
      "type": "missing",
      "loc": ["body", "amount"],
      "msg": "Field required"
    }
  ]
}
```

[Copiar](#)

Como tratar

Sempre trate o resultado pelo **HTTP status** e, em seguida, pelo `detail`. Para pagamentos com cartão, considere também o campo `code` do corpo (ACCEPTED/AUTHORIZED = aprovado).

Erros da adquirente

Quando a recusa vem da **adquirente** e não da PagMaxx, o `detail` é um objeto com o código e a mensagem originais dela:

```
{
  "detail": {
    "code": "MERCHANT_NOT_REGISTERED",
    "message": "PIX DESABILITADO PARA HABILITAR CONTATE CENTRAL DE ATENDIMENTO"
  }
}
```

[Copiar](#)

422 indica recusa da adquirente — por exemplo, meio de pagamento não habilitado para o estabelecimento. Nesse caso repetir a requisição não resolve: o cadastro precisa ser ajustado junto a adquirente. 502 indica adquirente indisponível ou resposta inválida, e a requisição pode ser repetida.

Códigos gerados pela PagMaxx

ACQUIRER_UNAVAILABLE (502) — falha de comunicação com a adquirente.

ACQUIRER_INVALID_RESPONSE (502) — a adquirente respondeu sem token transacional.

Status codes

STATUS	DETAIL	QUANDO OCORRE
200	OK	Requisição processada com sucesso.
201	Created	Recurso criado (ex.: assinatura).
400	Requisição inválida / regra de negócio	Ex.: 3DS obrigatório ausente, tipo de cancelamento incompatível.
401	Não autenticado	Token JWT inválido/expirado ou API key inválida/revogada.
403	Not authenticated / conta pendente	Header de autenticação ausente, ou conta ainda não aprovada.
404	Recurso não encontrado	Pagamento, link, assinatura ou transação inexistente.
409	Conflito de estado	Ex.: assinatura já autorizada / não está ativa.
422	Erro de validação / recusa da adquirente	Campos inválidos no corpo, cartão recusado na tokenização, ou recusa da adquirente (detail com code e message).
429	Muitas tentativas	Rate limit excedido (por IP, conta ou API key).
500	Erro interno	Falha inesperada ao processar a requisição.
502	Erro na integração com a adquirente	A adquirente não respondeu ou retornou erro.

Rate limiting

Vários endpoints têm limite de requisições por minuto. Ao exceder, a API responde 429. Limites por endpoint:

- /auth/token — 5/min · /auth/refresh — 20/min
- /payments/pix/sale e /payments/pix/get-sale — 5/min
- /payments/simulate-fee — 100/min
- Por **API Key** — limite dedicado por chave

Pagamentos recusados ≠ erros de API

Uma venda **recusada pelo emissor** não é um erro da API: o corpo da resposta é repassado pela adquirente com code diferente de ACCEPTED/AUTHORIZED, e o HTTP status reflete a decisão da adquirente. Trate esse caso na sua lógica de checkout, não como exceção de integração.

[← API Keys](#)

[Pagamentos →](#)

Criar uma venda

Produção

POST /api/payments/pay

Processa o pagamento de uma venda com cartão de crédito. Uma venda é a troca de um produto ou serviço por um valor monetário e pode ser revertida ou reembolsada via /payments/void.

A resposta é o retorno da adquirente acrescido do objeto _pagmaxx com os identificadores internos. O HTTP status reflete o resultado na adquirente.

Autenticação

Este endpoint usa **Bearer JWT** (Authorization). Se o seu estabelecimento exige 3DS obrigatório, use /payments/pay-secure, que também aceita autenticação por X-API-Key.

Exemplo de corpo (PAN aberto)

```
{
  "amount": 100.50,
  "currency": "BRL",
  "email": "user@example.com",
  "type": "CREDIT",
  "cardNumber": "4111111111111111",
  "cardSecurityCode": "123",
  "cardExpirationDate": "1229",
  "cardHolderName": "JOAO SILVA",
  "installments": 1
}
```

 Copiar

Headers

Authorization

Access token gerado em POST /auth/token.

Bearer eyJhbGciOiJIUzI1NiIsInR...

Parâmetros do Body

amount number **required**

O valor da transação.

100.50

currency text **required**

A moeda da transação.

BRL

email email

O endereço de e-mail do pagador.

user@example.com

type text

CREDIT ou DEBIT. Obrigatório apenas para transações de débito em e-commerce.

CREDIT

cardNumber text

Número do cartão (PAN). Obrigatório quando não se usa slugToken/slugStoredCard.

4111111111111111

cardSecurityCode text

CVV. Não é obrigatório para transações tokenizadas. Obrigatório na 1ª transação recorrente. Opcional para cartões armazenados.

123

cardExpirationDate text

Validade do cartão (MMYY). Obrigatória quando o PAN aberto é enviado.

1229

cardHolderName text

Nome do portador do cartão.

JOAO SILVA

orderId text

O identificador do pedido.

orderId

installments number

Número de parcelas. Cada parcela deve ser maior que R\$ 1,00.

installments

note text

Descrição sobre a transação.

note

slugToken text

String de 32 caracteres que representa um cartão tokenizado por POST /payments/tokenize-card. Substitui cardNumber, cardExpirationDate e cardSecurityCode.

slugToken

slugStoredCard text

String de 32 caracteres que representa um cartão armazenado no cofre (vault). Substitui cardNumber e cardExpirationDate.

slugStoredCard

BillingAddress

postalCode text

Código postal do endereço de cobrança.

postalCode

street text

Nome da rua e número.

street

complement text

Dados de endereço adicionais.

complement

phone text

Telefone associado ao endereço de cobrança.

phone

ThreeDsData

xid text

Identificador da transação. String base64 de tamanho 28.

xid

cavv text

CAVV (Visa/Elo) ou UCAF (Mastercard). String base64 de tamanho 32. Obrigatório para transações 3DS.

cavv

cavvResultCode text

Código de resultado do CAVV (string de tamanho 1).

cavvResultCode

secureVersion text

Versão do 3DS no formato x.y. Necessária em todas as transações 3DS.

secureVersion

directoryServerTransactionId text

ID de transação do Directory Server. 32 caracteres. Requerido apenas para Mastercard 3DS.

directoryServerTransactionId

threeDsServerTransactionId text

ID de transação do 3DS Server. 32 caracteres. Requerido apenas para Amex 3DS.

threeDsServerTransactionId

RecurringData

isFirstRecurring boolean

Indica se é o primeiro pagamento recorrente. Obrigatório apenas na 1ª transação recorrente.

true | false

brandReferenceId text

Identificador da transação recorrente original (mandato MIT). Recebido na 1ª resposta de venda; necessário nas cobranças seguintes.

PaymentSplit

splitType text

Tipo de split: PERCENTUAL ou ABSOLUTE.

splits

Lista de recebedores do split.

+ Adicionar item

Testar requisição

cURL

Node.js

Python

```
curl -X POST "https://api.prod.pagmaxx.com/api/payments/pay" \  
\  
  -H "Content-Type: application/json" \  
  -d '{}'
```

Copiar

Resposta:



```
// Aguardando requisição...
```

Parâmetros do corpo

PRINCIPAIS

amount number obrigatório	Valor da transação (ex.: 100.50).
currency string obrigatório	Moeda da transação (ex.: BRL).
type string opcional	CREDIT ou DEBIT. Obrigatório para débito em e-commerce.
email string opcional	E-mail do pagador.
orderId string opcional	Identificador do pedido no seu sistema.
installments number opcional	Número de parcelas. Cada parcela deve ser $\geq$ R\$ 1,00.
note string opcional	Observação livre da transação.

CARTÃO (OU TOKENIZAÇÃO)

cardNumber string opcional	PAN. Obrigatório se não usar slugToken/slugStoredCard.
cardExpirationDate string opcional	Validade MMY. Obrigatória com PAN aberto.
cardSecurityCode string opcional	CVV. Obrigatório na 1ª recorrente; opcional para cartão no cofre.
cardHolderName string opcional	Nome do portador.
slugToken string opcional	Cartão tokenizado (32 chars). Substitui o PAN + CVV.
slugStoredCard string opcional	Cartão no cofre (32 chars). Substitui o PAN.

BILLINGADDRESS (OBJETO)

billingAddress.postalCode string opcional	CEP do endereço de cobrança.
billingAddress.street string opcional	Rua e número.
billingAddress.complement string opcional	Complemento.
billingAddress.phone string opcional	Telefone de cobrança.

THREEDSDATA (OBJETO) — AUTENTICAÇÃO 3DS JÁ REALIZADA

threeDsData.cavv string opcional	CAVV/UCAF (base64, 32). Obrigatório em transações 3DS.
threeDsData.xid string opcional	XID (base64, 28).
threeDsData.cavvResultCode string opcional	Código de resultado do CAVV.
threeDsData.secureVersion string opcional	Versão 3DS (x.y).
threeDsData.directoryServerTransactionId string opcional	32 chars. Apenas Mastercard.
threeDsData.threeDsServerTransactionId string opcional	32 chars. Apenas Amex.

RECURRINGDATA E PAYMENTSPLIT (OBJETOS)

recurringData.isFirstRecurring boolean opcional	Marca a 1ª transação recorrente.
recurringData.brandReferenceId string opcional	Mandato MIT para as cobranças seguintes.
paymentSplit.splitType string opcional	PERCENTUAL ou ABSOLUTE.
paymentSplit.splits[] array opcional	Lista de { documentId, value } dos recebedores.

Resposta

O corpo é o retorno da adquirente acrescido de `_pagmaxx`. O campo `code` indica o resultado: `ACCEPTED/AUTHORIZED` = aprovado; qualquer outro = recusado.

● 200 — Aprovada

```
{
  "code": "ACCEPTED",
  "muid": "1EC0F4A2B3C4D5E6F7A8B9C0D1E2F3A4",
  "brand": "VISA",
  "lastDigits": "1111",
  "amount": 100.50,
  "_pagmaxx": {
    "payment_id": "4f9c1d2e-7a3b-4c5d-8e9f-0a1b2c3d4e5f",
    "brand": "VISA",
    "payer_name": "JOAO SILVA",
    "origin": "ONLINE",
    "card_last4": "4321"
  }
}
```

[Copiar](#)

● Recusada pela adquirente (status e campos repassados na resposta)

```
{
  "code": "DENIED",
  "msg": "Transacao nao autorizada pelo emissor",
  "_pagmaxx": {
    "payment_id": "4f9c1d2e-7a3b-4c5d-8e9f-0a1b2c3d4e5f",
    "brand": "VISA",
    "payer_name": "JOAO SILVA",
    "origin": "ONLINE",
    "card_last4": "4321"
  }
}
```

[Copiar](#)

❗ lastDigits é o final do token, não do cartão

A adquirente tokeniza o cartão antes de processar, e o `lastDigits` devolvido corresponde aos 4 últimos dígitos do **token** — não do cartão que o portador digitou. Não use esse valor para identificar o cartão para o pagador: ele não reconhece o número. Use o `_pagmaxx.card_last4`, que traz os 4 últimos do cartão realmente digitado. Ele vem `null` quando a venda não passou pela nossa tokenização.

💡 Os campos fora de `_pagmaxx` são repassados integralmente pela adquirente e podem variar por bandeira e cenário. Sempre trate o resultado pelo HTTP status e por code.

Erros

STATUS	DETAIL	QUANDO OCORRE
400	Autenticacao 3DS obrigatoria para pagamentos com cartao.	EC com 3DS obrigatório e sem dados de 3DS aprovado.
400	Autenticacao 3DS rejeitada pelo emissor.	3DS reprovado para o EC com 3DS obrigatório.
403	Not authenticated	Header Authorization ausente ou token inválido.
500	<mensagem do erro>	Falha interna ao processar a venda.

[← Erros & Status](#)

[Pagamento com 3DS obrigatório →](#)

Pagamento com 3DS obrigatório

Produção



POST /api/payments/pay-secure

Idêntico a /payments/pay, porém **exige autenticação 3DS aprovada** (challenge) antes de processar a venda. Vendas sem 3DS aprovado são bloqueadas com 400. É a rota recomendada para integrações que precisam transferir a responsabilidade de fraude ao emissor e reduzir chargebacks.

Autenticação dupla

Aceita **X-API-Key** (server-to-server) **ou** Authorization: Bearer <JWT>. Envie apenas um dos dois no testador.

Exemplo de corpo

```
{
  "amount": 100.50,
  "currency": "BRL",
  "type": "CREDIT",
  "slugStoredCard": "9F3A2B1C4D5E6F7A8B9C0D1E2F3A4B5C",
  "cardSecurityCode": "123",
  "threeDsData": {
    "cavv": "AAABBJg0VhI0VniQEjRWAAAAA=",
    "secureVersion": "2.2.0",
    "directoryServerTransactionId": "f38e6948-..."
  }
}
```

 Copiar

Headers

X-API-Key

API Key server-to-server (alternativa ao Authorization).

pmx_live_...

Authorization

Access token JWT (alternativa ao X-API-Key).

Bearer eyJhbG...

Parâmetros do Body

amount number **required**

Valor da transação.

currency text **required**

Moeda da transação.

type text

CREDIT ou DEBIT.

slugStoredCard text

Cartão no cofre (ou use cardNumber / slugToken).

cardSecurityCode text

CVV.

ThreeDsData

cavv text

CAVV/UCAF retornado no 3DS. Obrigatório.

secureVersion text

Versão do 3DS.

directoryServerTransactionId text

DS Transaction ID (Mastercard).

xid text

XID quando aplicável.

[Testar requisição](#)[cURL](#)[Node.js](#)[Python](#)

```
curl -X POST "https://api.prod.pagmaxx.com/api/payments/pay-secure" \  
\  
-H "Content-Type: application/json" \  
-d '{}'
```

[Copiar](#)**Resposta:** ---

```
// Aguardando requisição...
```

Pré-requisito: autenticação 3DS

Antes de chamar esta rota, execute o fluxo 3DS ([setup](#) → [authentication](#) → [challenge result](#)) e envie o resultado no objeto `threeDsData`. Quando o `cavv` é enviado, a PagMaxx valida que existe uma autenticação **ACCEPTED** correspondente — protegendo contra reuso/spoofing.

Parâmetros do corpo

Os campos do corpo são os mesmos de </payments/pay>. A diferença é que `threeDsData` com `cavv` aprovado é obrigatório.

<code>amount</code> number obrigatório	Valor da transação.
<code>currency</code> string obrigatório	Moeda (ex.: BRL).
<code>type</code> string opcional	CREDIT ou DEBIT.
<code>cardNumber</code> / <code>slugToken</code> / <code>slugStoredCard</code> string opcional	Forma de pagamento: PAN aberto, cartão tokenizado ou cartão no cofre.
<code>threeDsData.cavv</code> string obrigatório	CAVV/UCAF da autenticação 3DS aprovada.
<code>threeDsData.secureVersion</code> string opcional	Versão do 3DS (ex.: 2.2.0).
<code>threeDsData.directoryServerTransactionId</code> string opcional	DS Transaction ID (Mastercard).

Resposta

Em caso de sucesso, a resposta é idêntica à de `/payments/pay`: retorno da adquirente acrescido de `_pagmaxx`.

● 200 — Aprovada

```
{
  "code": "ACCEPTED",
  "muid": "1EC0F4A2B3C4D5E6F7A8B9C0D1E2F3A4",
  "brand": "VISA",
  "lastDigits": "1111",
  "_pagmaxx": {
    "payment_id": "4f9c1d2e-7a3b-4c5d-8e9f-0a1b2c3d4e5f",
    "brand": "VISA",
    "payer_name": "JOAO SILVA",
    "origin": "ONLINE",
    "card_last4": "4321"
  }
}
```

 Copiar

lastDigits é o final do token, não do cartão

A adquirente tokeniza o cartão antes de processar, e o `lastDigits` devolvido corresponde aos 4 últimos dígitos do **token** — não do cartão que o portador digitou. Não use esse valor para identificar o cartão para o pagador: ele não reconhece o número. Use o `_pagmaxx.card_last4`, que traz os 4 últimos do cartão realmente digitado. Ele vem `null` quando a venda não passou pela nossa tokenização.

Erros

STATUS	DETAIL	QUANDO OCORRE
400	Autenticacao 3DS obrigatoria para pagamentos com cartao.	Requisição sem threeDsData.
400	Autenticacao 3DS nao encontrada. Refaca o pagamento.	cavv enviado sem autenticação correspondente registrada.
400	Autenticacao 3DS rejeitada pelo emissor.	A autenticação 3DS não foi aprovada (≠ ACCEPTED).
401	API key invalida ou revogada.	X-API-Key inválida.
403	Not authenticated	Sem X-API-Key e sem Authorization.
500	<mensagem do erro>	Falha interna ao processar a venda.

[< Pagamentos](#)

[Cancelar Transação >](#)

Cancelar transação

Produção

POST /api/payments/void

Processa o cancelamento de uma transação. É possível cancelar tanto uma **pré-autorização** quanto uma **venda**. Ao cancelar uma pré-autorização, o limite reservado é liberado; ao cancelar uma venda, o valor é reembolsado.

Use `PRE_AUTHORIZATION` como `type` para cancelar uma pré-autorização e `CREDIT` para cancelar uma venda. O `type` deve corresponder ao tipo da transação original.

A resposta é o retorno da adquirente acrescido do objeto `_pagmaxx` com os identificadores internos. O HTTP status reflete o resultado na adquirente.

Autenticação

Este endpoint usa **Bearer JWT** (Authorization). O token é gerado em `POST /auth/token`.

Cancelamento D+N

Transações realizadas há mais de um dia geram um cancelamento **D+N**: o reembolso retorna com status de ciclo `PENDING`. Nesse caso você pode informar um identificador próprio em `refundTrackingNumber` (UUID v4 de 32 caracteres em hexadecimal, sem hífen).

Exemplo de corpo

```
{
  "type": "PRE_AUTHORIZATION",
  "amount": 100.50,
  "currency": "BRL",
  "muid": "1234567890",
  "trackingNumber": "987654321",
  "originalRrn": "111222333",
  "email": "user@example.com",
  "authorizationCode": 123456,
  "refundTrackingNumber": "b9bb06c9f71c42658b2c644c39f155f7"
}
```

 Copiar

Headers

Authorization

Deve ser um Access token gerado a partir de suas credenciais no `/auth/token`.

Bearer eyJhbGciOiJIUzI1NiIsInR...

Parâmetros do Body

type text **required**

Tipo de cancelamento.

PRE_AUTHORIZATION

amount number **required**

Valor da transação.

100.50

currency text **required**

Moeda da transação.

BRL

muid text **required**

Identificador único da transação original.

MUID da transação original

trackingNumber text **required**

Número de rastreio da transação original.

Número de rastreio da transação original

originalRrn text **required**

Número de referência da transação original.

RRN da transação original

email email

Email do cliente.

user@example.com

authorizationCode number

Código de autorização da transação.

refundTrackingNumber text

Opcional, apenas para cancelamentos D+N. UUID v4 de 32 caracteres sem hífen.

Testar requisição

cURL

Node.js

Python

```
curl -X POST "https://api.prod.pagmaxx.com/api/payments/void" \  
\  
  -H "Content-Type: application/json" \  
  -d '{}'
```

Copiar

Resposta:

● ---

```
// Aguardando requisição...
```

Parâmetros do corpo

type string obrigatório	PRE_AUTHORIZATION ou CREDIT. Deve corresponder ao tipo da transação original.
amount number obrigatório	Valor da transação.
currency string obrigatório	Moeda da transação (ex.: BRL).
muid string obrigatório	Identificador único da transação original.
trackingNumber string obrigatório	Número de rastreio da transação original.
originalRrn string obrigatório	Número de referência (RRN) da transação original.
email string opcional	E-mail do cliente.
authorizationCode number opcional	Código de autorização da transação original.
refundTrackingNumber string opcional	Apenas para cancelamentos D+N. UUID v4 em hexadecimal, 32 caracteres, sem hífen.

Resposta

O corpo é o retorno da adquirente acrescido de `_pagmaxx`. Os campos fora de `_pagmaxx` são repassados pela adquirente e podem variar por cenário.

● 200 — Cancelada

```
{
  "code": "ACCEPTED",
  "muid": "1EC0F4A2B3C4D5E6F7A8B9C0D1E2F3A4",
  "_pagmaxx": {
    "payment_id": "4f9c1d2e-7a3b-4c5d-8e9f-0a1b2c3d4e5f",
    "brand": "VISA",
    "payer_name": "JOAO SILVA",
    "origin": "ONLINE"
  }
}
```

[Copiar](#)

Erros

STATUS	DETAIL	QUANDO OCORRE
404	Pagamento não encontrado	O muid não corresponde a uma venda do estabelecimento.
400	Tipo de pagamento incompatível para cancelamento	O type difere do tipo da transação original.
500	Erro ao cancelar pagamento: <mensagem>	Falha interna ao processar o cancelamento.

[← Pagamento com 3DS obrigatório](#)[Criar Link de Pagamento →](#)

Criar Link de Pagamento

Produção

POST /api/payment-link/create

Cria um link de pagamento por cartão de crédito. O cliente final acessa a URL pública do link e conclui o pagamento sem que você precise integrar o fluxo de venda diretamente.

A resposta é o retorno da adquirente com os dados do link criado — incluindo o slug, a URL pública e o status inicial PENDING.

Autenticação

Este endpoint usa **Bearer JWT** (Authorization). O token é gerado em `POST /auth/token`.

Este endpoint não faz split

O link criado aqui usa o checkout da adquirente, que não divide o valor entre parceiros. Enviar `split_config` aqui retorna 422. Para split, use [POST /api/payment-links](#), que cria o link no checkout PagMaxx.

Valor mínimo da parcela

Em parcelamentos, cada parcela individual deve ser de no mínimo **R\$ 1,00**. Caso contrário a criação é rejeitada com 422.

Exemplo de corpo

```
{
  "linkName": "Pagamento Serviço",
  "totalAmount": "150.00",
  "dtExpiration": "2025-12-05T18:00:00Z",
  "productType": "CREDIT",
  "installments": 1,
  "shoppingItems": [
    { "name": "Item A", "quantity": 1, "amount": "50.00" },
    { "name": "Item B", "quantity": 2, "amount": "50.00" }
  ]
}
```

 Copiar

Headers

Authorization

Deve ser um Access token gerado a partir de suas credenciais no `/auth/token`.

Bearer eyJhbGciOiJIUzI1NiIsInR...

Parâmetros do Body

linkName text **required**

Nome do link de pagamento.

totalAmount text **required**

Valor total do link.

dtExpiration text **required**

Data de expiração do link.

productType text **required**

Tipo de produto. Permitido: CREDIT

installments number **required**

Quantidade máxima de parcelas aceitas.

ShoppingItems

shoppingItems

Lista detalhada dos itens que compõem o link. [OBJETO]

+ Adicionar item

Testar requisição

cURL

Node.js

Python

```
curl -X POST "https://api.prod.pagmaxx.com/api/payment-link/create" \  
\  
-H "Content-Type: application/json" \  
-d '{}'
```

Copiar

Resposta:

● ---

```
// Aguardando requisição...
```

Parâmetros do corpo

PRINCIPAIS

<code>linkName</code> string obrigatório	Nome do link de pagamento. Máximo de 80 caracteres.
<code>totalAmount</code> string obrigatório	Valor total do link (ex.: "150.00").
<code>dtExpiration</code> string obrigatório	Data/hora de expiração no formato ISO (ex.: "2025-12-05T18:00:00Z").
<code>productType</code> string obrigatório	Tipo de produto. Permitido: CREDIT.
<code>installments</code> number obrigatório	Quantidade máxima de parcelas aceitas (1 a 21).

SHOPPINGITEMS[] (ARRAY DE OBJETOS)

<code>shoppingItems[].name</code> string opcional	Descrição do item. Máximo de 80 caracteres.
<code>shoppingItems[].quantity</code> number opcional	Quantidade do item (1 a 9999).
<code>shoppingItems[].amount</code> string opcional	Valor unitário do item (ex.: "50.00").

Resposta

O corpo é o retorno da adquirente com os dados do link criado. Os campos abaixo são repassados pela adquirente e podem variar.

● 201 — Link criado

```
{
  "slug": "1EB94234542C40A1B2C3D4E5F6A7B8C9",
  "linkName": "Pagamento Serviço",
  "url": "https://pay.pagmaxx.com/1/1EB94234542C40A1B2C3D4E5F6A7B8C9",
  "status": "PENDING",
  "totalAmount": "150.00",
  "productType": "CREDIT",
  "installments": 1,
  "dtExpiration": "2025-12-05T18:00:00Z"
}
```

[Copiar](#)

Erros

STATUS	DETAIL	QUANDO OCORRE
422	Parcela individual nao pode ser inferior a R\$ 1,00 ...	O valor de cada parcela fica abaixo de R\$ 1,00 para o número de parcelas informado.
500	Erro ao criar link de pagamento na PagMaxx	Falha interna ao criar o link na adquirente.

[← Cancelar Transação](#)[Obter Link de Pagamento →](#)

Consultar Links de Pagamento

Produção



GET

/api/payment-link/get

Lista ou consulta links de pagamento. Todos os filtros são enviados como **parâmetros de query** e são opcionais: sem filtros, o endpoint retorna os links do estabelecimento.

A resposta é o retorno da adquirente com a lista de links que atendem aos filtros informados.

Autenticação

Este endpoint usa **Bearer JWT** (Authorization). O token é gerado em POST /auth/token.

Operadores de filtro

Os sufixos `__like`, `__goe` (maior ou igual) e `__loe` (menor ou igual) permitem buscas aproximadas e por faixa em nome, data e valor.

Paginação e ordenação

Use `limit` (padrão 20, **máximo 1000**) e `offset` para paginar; a resposta traz `meta.total_count` com o total. Para ordenar use `order_by` — ex.: `-dtInsert` (mais recentes) ou `dtInsert` (mais antigos). Atenção: o parâmetro é `order_by` (com underline), não `ordering`.

Headers

Authorization

Deve ser um Access token gerado a partir de suas credenciais no `/auth/token``.

Bearer eyJhbGciOiJIUzI1NiIsInR...

Query Parâmetros

slug text

Identificador único do Payment Link.

UUID do link (opcional)

slugMerchant text

Identificador único do merchant.

UUID do merchant

paymentLinkStatus text

Filtrar por status do link. PENDING | PAID | EXPIRED | CANCELED

paymentLinkStatus

linkName_like text

Busca flexível pelo nome (contém).

buscar por parte do nome

linkName text

Busca exata pelo nome do link.

nome exato

dtInsert text

Busca pela data exata de criação.

dtInsert

dtInsert_goe text

Links criados após esta data.

dtInsert_goe

dtInsert_loe text

Links criados antes desta data.

dtInsert_loe

totalAmount text

Valor total exato.

totalAmount_goe text

Valor mínimo.

totalAmount_lo text

Valor máximo.

limit text

Máximo por página (padrão 20, máx 1000).

offset text

Registro inicial (padrão 0).

order_by text

Ordenação: -dtInsert (recentes) / dtInsert (antigos).

Testar requisição

cURL

Node.js

Python

```
curl -X GET "https://api.prod.pagmaxx.com/api/payment-link/get" \  
\  
-H "Content-Type: application/json" \  
-d '{}'
```

Copiar

Resposta:



```
// Aguardando requisição...
```

Parâmetros do corpo

FILTROS (QUERY)

slug string opcional	Identificador único do Payment Link.
slugMerchant string opcional	Identificador único do merchant.
paymentLinkStatus string opcional	Status do link: PENDING PAID EXPIRED CANCELED.
linkName string opcional	Busca exata pelo nome do link.
linkName__like string opcional	Busca flexível pelo nome (contém).
dtInsert string opcional	Data exata de criação.
dtInsert__goe string opcional	Criados a partir desta data (maior ou igual).
dtInsert__loe string opcional	Criados até esta data (menor ou igual).
totalAmount string opcional	Valor total exato.
totalAmount__goe string opcional	Valor mínimo (maior ou igual).
totalAmount__loe string opcional	Valor máximo (menor ou igual).
limit number opcional	Máximo de registros por página. Padrão 20, máximo 1000.
offset number opcional	Registro inicial da página. Padrão 0. Ex.: offset=20 pula os 20 primeiros.

order_by
string opcional

Ordenação. Ex.: -dtInsert (mais recentes) ou dtInsert (mais antigos); vírgula p/ múltiplos campos.

Resposta

O corpo é o retorno da adquirente com a lista de links. Os campos abaixo são repassados pela adquirente e podem variar.

● 200 — Lista de links

```
{
  "meta": { "limit": 20, "offset": 0, "total_count": 1520 },
  "objects": [
    {
      "slug": "1EB94234542C40A1B2C3D4E5F6A7B8C9",
      "linkName": "Pagamento Serviço",
      "linkUrl": "https://pay.pagmaxx.com/1/1EB94234542C40A1B2C3D4E5F6A7B8C9",
      "paymentLinkStatus": "PENDING",
      "totalAmount": 150.0,
      "installments": 1,
      "dtExpiration": "2025-12-05T18:00:00Z",
      "dtInsert": "2025-11-20T14:30:00Z"
    }
  ]
}
```

 Copiar

Erros

STATUS	DETAIL	QUANDO OCORRE
422	Parâmetro de paginação inválido	limit fora de 1..1000 ou offset negativo.
400	Filtro/ordenação não permitido pela adquirente	order_by com campo inválido ou parâmetro não suportado.
500	Erro ao recuperar link de pagamento na PagMaxx	Falha interna ao consultar os links na adquirente.

← Criar Link de Pagamento

Atualizar Link de Pagamento →

1EB94234542C40...

Parâmetros do Body

linkName text

Nome do link de pagamento.

Descrição do link

totalAmount text

Valor total do link.

100.50

dtExpiration text

Data de expiração do link.

102029

installments number

Quantidade máxima de parcelas aceitas.

1

ShoppingItems

shoppingItems

Lista detalhada dos itens que compõem o link. [OBJETO]

+ Adicionar item

Testar requisição

cURL

Node.js

Python

```
curl -X PUT "https://api.prod.pagmaxx.com/api/payment-link/update/{slug"
\
-H "Content-Type: application/json" \
-d '{}'
```

[Copiar](#)

Resposta:

● ---

```
// Aguardando requisição...
```

Parâmetros do corpo

CAMINHO (PATH)

slug string obrigatório	Identificador único do link de pagamento. Informado na URL.
--	---

CORPO (TODOS OPCIONAIS)

linkName string opcional	Nome do link de pagamento.
totalAmount string opcional	Valor total do link (ex.: "150.00").
dtExpiration string opcional	Data/hora de expiração no formato ISO (ex.: "2025-12-05T18:00:00Z").
installments number opcional	Quantidade máxima de parcelas aceitas (1 a 21).

SHOPPINGITEMS[] (ARRAY DE OBJETOS)

<code>shoppingItems[].name</code> string opcional	Descrição do item. Máximo de 80 caracteres.
<code>shoppingItems[].quantity</code> number opcional	Quantidade do item (1 a 9999).
<code>shoppingItems[].amount</code> string opcional	Valor unitário do item (ex.: "50.00").

Resposta

O corpo é o retorno da adquirente com os dados do link atualizado. Os campos abaixo são repassados pela adquirente e podem variar.

● 200 — Link atualizado

```
{
  "slug": "1EB94234542C40A1B2C3D4E5F6A7B8C9",
  "linkName": "Pagamento Serviço",
  "url": "https://pay.pagmaxx.com/1/1EB94234542C40A1B2C3D4E5F6A7B8C9",
  "status": "PENDING",
  "totalAmount": "150.00",
  "installments": 1,
  "dtExpiration": "2025-12-05T18:00:00Z"
}
```

[Copiar](#)

Erros

STATUS	DETAIL	QUANDO OCORRE
422	Parcela individual nao pode ser inferior a R\$ 1,00 ...	O valor de cada parcela fica abaixo de R\$ 1,00 para o número de parcelas informado.
500	Erro ao atualizar link de pagamento na PagMaxx	Falha interna ao atualizar o link na adquirente.

[← Obter Link de Pagamento](#)[Venda Pix →](#)

Criar Link com Checkout PagMaxx (com split)

Produção

POST /api/payment-links

Cria um link de pagamento hospedado no **checkout PagMaxx**. A resposta traz a `public_url` para você enviar ao pagador, que conclui a compra sem que você precise integrar o fluxo de venda.

É neste endpoint que existe o **split de pagamento**: a divisão automática do valor entre você e seus parceiros cadastrados.

Qual endpoint de link usar

Existem dois. Este (/api/payment-links) usa o checkout PagMaxx e **aceita split**. O endpoint [/api/payment-link/create](#) usa o checkout da adquirente e **não** aceita split — se você enviar `split_config` nele, a requisição é recusada com 422.

Split é exclusivo do cartão de crédito

O split exige `card_mode: "credit"`. Todos os parceiros incluídos precisam estar com o **credenciamento ativo** — a validação é feita na hora da criação e o link não é criado se algum parceiro estiver pendente.

Exemplo de corpo — com split percentual

```
{
  "amount": 100.00,
  "title": "Consultoria - Agosto",
  "installments": 1,
  "card_mode": "credit",
  "require_3ds": true,
  "split_config": {
    "split_type": "PERCENTUAL",
    "items": [
      { "partner_id": "5ca18d4a-4a92-4b45-8c0a-9bf6f490cb85", "value": 30 }
    ]
  }
}
```

 Copiar

Exemplo de corpo — sem split

```
{
  "amount": 250.00,
  "title": "Mensalidade",
  "installments": 3,
  "card_mode": "credit"
}
```

[Copiar](#)

Headers

Authorization

Access token gerado a partir das suas credenciais em `/auth/token`.

Bearer {access_token}

Parâmetros do Body

amount text **required**

Valor total do link, em reais.

100.00

title text

Título exibido no checkout.

Consultoria — Agosto

description text

Descrição exibida no checkout.

Descrição exibida ao pagador

installments number

Número máximo de parcelas no crédito (1 a 21).

1

card_mode text

credit (à vista ou parcelado) ou debit (somente à vista).

credit

require_3ds text

Exige autenticação 3DS no checkout. Padrão: true.

expires_at text

Data/hora de expiração (ISO 8601). Sem valor, o link não expira.

Split_config**split_config.split_type** text

PERCENTUAL ou ABSOLUTE. Obrigatório quando há split.

split_config.items[0].partner_id text

ID do parceiro que recebe a parte. Obtido em /api/partners.

split_config.items[0].value text

Percentual (ex.: 30 = 30%) ou valor absoluto em reais.

[Testar requisição](#)[cURL](#)[Node.js](#)[Python](#)

```
curl -X POST "https://api.prod.pagmaxx.com/api/payment-links/" \
-H "Content-Type: application/json" \
-d '{}'
```

[Copiar](#)

Resposta:



```
// Aguardando requisição...
```

Parâmetros do corpo

PRINCIPAIS

amount number obrigatório	Valor total do link, em reais (ex.: 100.00). Deve ser maior que zero.
title string opcional	Título exibido no checkout. Máximo de 120 caracteres.
description string opcional	Descrição exibida no checkout. Máximo de 500 caracteres.
expires_at string opcional	Data/hora de expiração em ISO 8601. Sem valor, o link não expira.
installments number opcional	Número máximo de parcelas no crédito, de 1 a 21. Padrão: 1.
card_mode string opcional	credit (à vista ou parcelado) ou debit (somente à vista). Padrão: credit.
require_3ds boolean opcional	Exige autenticação 3DS no checkout. Padrão: true.
split_config object opcional	Divisão do valor entre parceiros. Somente no crédito.

SPLIT_CONFIG (OBJETO)

<code>split_config.source_split_rule_id</code> string opcional	ID de uma regra de split salva, obtido em GET /api/split-rules. Sozinho, já aplica os participantes e o <code>split_type</code> da regra — não é preciso enviar items.
<code>split_config.split_type</code> string opcional	PERCENTUAL (value em %) ou ABSOLUTE (value em reais). Obrigatório quando você envia items; usando apenas a regra salva, vem da regra.
<code>split_config.items[]</code> array opcional	De 1 a 20 participantes. Não pode repetir o mesmo parceiro. Obrigatório no split livre; opcional quando você usa <code>source_split_rule_id</code> .
<code>split_config.items[].partner_id</code> string obrigatório	ID do parceiro, obtido em GET /api/partners. Precisa estar com credenciamento ativo.
<code>split_config.items[].value</code> number obrigatório	Parte do parceiro. Em PERCENTUAL, 30 significa 30%. Em ABSOLUTE, é o valor em reais.

Duas formas de enviar o split

Regra salva: envie apenas { "split_config": { "source_split_rule_id": "<uuid>" } }. Os participantes e o `split_type` vêm da regra cadastrada no portal.

Split livre: envie `split_type` e `items`. Enviando os dois, `items` prevalece e a regra fica registrada apenas como origem — a regra salva nunca é alterada.

Os campos de split ficam dentro de split_config

Enviar `split_type`, `items` ou `source_split_rule_id` na raiz do corpo retorna 422. Esses campos existem somente dentro do objeto `split_config`.

Quanto sobra para você

A soma dos participantes deve ser menor que o total: o restante fica com você, o estabelecimento principal. Em PERCENTUAL a soma deve ser inferior a 100; em ABSOLUTE, inferior ao `amount`. Você não entra na lista de participantes.

Resposta

● 201 — Link criado

```
{
  "id": "7847cd95-ae0a-4a95-91ba-2a7434666345",
  "public_id": "8ae0ef7d-3701-4c0f-a450-aaaacae26ab7",
  "public_url": "https://portal.pagmaxx.com/link/8ae0ef7d-3701-4c0f-a450-aaaacae26ab7",
  "amount": 100.0,
  "currency": "BRL",
  "status": "pending",
  "installments": 1,
  "card_mode": "credit",
  "require_3ds": true,
  "title": "Consultoria - Agosto",
  "description": null,
  "expires_at": null,
  "created_at": "2026-08-12T12:10:29.290608",
  "split": {
    "split_type": "PERCENTUAL",
    "source_split_rule_id": null,
    "items": [
      {
        "partner_id": "5ca18d4a-4a92-4b45-8c0a-9bf6f490cb85",
        "partner_name": "Parceiro Exemplo",
        "partner_document_id": "63215815000108",
        "value": 30.0
      }
    ]
  }
}
```

 Copiar

Consultar e cancelar

O mesmo recurso responde à consulta e ao cancelamento do link, sempre restrito aos links do seu próprio estabelecimento.

Outros métodos

```
GET /api/payment-links/ # lista seus links (page, limit,
GET /api/payment-links/{public_id} # consulta um link
POST /api/payment-links/{public_id}/deactivate # cancela um link ainda pendente
```

 Copiar

Erros

STATUS	DETAIL	QUANDO OCORRE
422	Split disponivel apenas em cartao de credito.	O split foi enviado com card_mode diferente de "credit".
422	Parceiro '{nome}' nao esta com credenciamento ativo na PagMaxx ...	Algum parceiro do split ainda não concluiu o credenciamento.
422	Voce nao pode incluir seu proprio CNPJ/CPF (regra OWNER_IN_SPLIT).	O documento do próprio estabelecimento foi incluído como participante.
422	Parcela individual nao pode ser inferior a R\$ 1,00 ...	O valor de cada parcela fica abaixo de R\$ 1,00 para o número de parcelas informado.
422	Extra inputs are not permitted	Campo desconhecido no corpo. Confira a grafia — o corpo não aceita campos fora da tabela acima.
422	Soma dos participantes ...	A soma do split atinge ou ultrapassa o total do link.
503	Nao foi possivel validar o credenciamento do parceiro ...	Indisponibilidade momentânea ao validar o credenciamento. Tente novamente.
404	Link de pagamento nao encontrado	O public_id não existe ou não pertence ao seu estabelecimento.

← Atualizar Link de Pagamento

Venda - Pix →

Venda Pix

Produção

POST /api/payments/pix/sale

Cria uma venda Pix e retorna o código **copia-e-cola** (qr_code/qrText) que deve ser exibido ao pagador. A venda permanece com status PENDING até a confirmação do pagamento.

A resposta é o retorno do Pix **normalizado** pela PagMaxx, acrescido do objeto `_pagmaxx` com os identificadores internos.

Autenticação

Este endpoint usa **Bearer JWT** (Authorization), gerado em POST /auth/token.

 Rate limit

Este endpoint aceita no máximo **5 requisições por minuto**. Ao exceder esse limite, a API responde HTTP 429 (Rate limit exceeded).

Exemplo de corpo

```
{
  "amount": "100.00",
  "currency": "BRL",
  "receiptId": "202512041200001",
  "payee": {
    "zipCode": "01311000",
    "city": "São Paulo",
    "state": "SP",
    "address": "Av Paulista, 1000"
  }
}
```

 Copiar

Headers

Authorization

Access token gerado em POST /auth/token.

Bearer eyJhbGciOiJIUzI1NiIsInR...

Parâmetros do Body

amount text required
Valor da transação Pix.

currency text **required**

Moeda da transação.

receiptId text **required**

ID do recibo no seu sistema.

Payee

zipCode text **required**

CEP do recebedor.

city text **required**

Cidade do recebedor.

state text **required**

UF do recebedor.

address text **required**

Endereço do recebedor.

Testar requisição

cURL

Node.js

Python

```
curl -X POST "https://api.prod.pagmaxx.com/api/payments/pix/sale" \  
\  
-H "Content-Type: application/json" \  
-d '{}'
```

Copiar

Resposta:

● ---

```
// Aguardando requisição...
```

Parâmetros do corpo

PRINCIPAIS

amount number string obrigatório	Valor da transação Pix (ex.: 100.00).
currency string opcional	Moeda da transação. Padrão: BRL.
receiptId string obrigatório	ID do recibo no seu sistema.

PAYEE (OBJETO)

<code>payee.zipCode</code> string obrigatório	CEP do recebedor.
<code>payee.city</code> string obrigatório	Cidade do recebedor.
<code>payee.state</code> string obrigatório	UF do recebedor.
<code>payee.address</code> string obrigatório	Endereço do recebedor.

Resposta

O corpo é o retorno do Pix normalizado e acrescido de `_pagmaxx`. O campo `qr_code` (também repetido em `qrText`) é o **copia-e-cola Pix**; o status permanece PENDING até o pagamento.

● 200 — Venda criada

```
{
  "id": "1EC0...",
  "qr_code": "00020126...BR.GOV.BCB.PIX...6304ABCD",
  "status": "PENDING",
  "amount": 100.00,
  "currency": "BRL",
  "expires_in_minutes": 30,
  "receipt_id": "202512041200001",
  "processing_date": "2026-06-17",
  "idTx": "1EC0...",
  "qrText": "00020126...6304ABCD",
  "loc": null,
  "locId": null,
  "payee": {
    "zipCode": "01311000",
    "city": "São Paulo",
    "state": "SP",
    "address": "Av Paulista, 1000"
  },
  "_pagmaxx": {
    "payment_id": "uuid",
    "brand": "pix",
    "payer_name": null,
    "origin": "ONLINE"
  }
}
```

 Copiar

💡 Guarde o idTx (igual a id) para consultar o status depois em GET `/payments/pix/get-sale/{purchase_id}`.

Erros

STATUS	DETAIL	QUANDO OCORRE
429	Rate limit exceeded	Mais de 5 requisições no intervalo de 1 minuto.
403	Not authenticated	Header Authorization ausente ou token inválido.

← Atualizar Link de Pagamento

Obter Venda Pix →

Obter Venda Pix

Produção

GET /api/payments/pix/get-sale/{purchase_id}

Consulta o status atual de uma venda Pix a partir do `purchase_id` — o `idTx/id` retornado na criação da venda. O status pode evoluir de `PENDING` para `PAID`, entre outros.

A resposta é o mesmo shape **normalizado** da venda Pix, acrescido de `_pagmaxx`.

Autenticação

Este endpoint usa **Bearer JWT** (Authorization), gerado em `POST /auth/token`.

Rate limit

Este endpoint aceita no máximo **5 requisições por minuto**. Ao exceder esse limite, a API responde `HTTP 429` (`Rate limit exceeded`).

Headers

Authorization

Access token gerado em `POST /auth/token`.

Bearer eyJhbGciOiJIUzI1NiIs...

Path Parâmetros

ID da Transação (IDTX)

ID (`idTx`) retornado na criação da venda Pix.

ID da Transação (IDTX)

Testar requisição

cURL

Node.js

Python

```
curl -X GET "https://api.prod.pagmaxx.com/api/payments/pix/get-sale" \
  \
  -H "Content-Type: application/json" \
  -d '{}'
```

Copiar

Resposta:

● ---

```
// Aguardando requisição...
```

Parâmetros

PATH

purchase_id

ID da transação Pix (idTx/id) retornado na criação.

string **obrigatório**

Resposta

Mesmo shape normalizado da venda Pix, acrescido de `_pagmaxx`. O status reflete a situação atual da cobrança (no exemplo abaixo, já PAID).

● 200 — Venda paga

```
{
  "id": "1EC0...",
  "qr_code": "00020126...BR.GOV.BCB.PIX...6304ABCD",
  "status": "PAID",
  "amount": 100.00,
  "currency": "BRL",
  "expires_in_minutes": 30,
  "receipt_id": "202512041200001",
  "processing_date": "2026-06-17",
  "idTx": "1EC0...",
  "qrText": "00020126...6304ABCD",
  "loc": null,
  "locId": null,
  "payee": {
    "zipCode": "01311000",
    "city": "São Paulo",
    "state": "SP",
    "address": "Av Paulista, 1000"
  },
  "_pagmaxx": {
    "payment_id": "uuid",
    "brand": "pix",
    "payer_name": null,
    "origin": "ONLINE"
  }
}
```

[Copiar](#)

💡 Use o status para decidir o fluxo: PENDING = aguardando pagamento; PAID = confirmado.

Erros

STATUS	DETAIL	QUANDO OCORRE
429	Rate limit exceeded	Mais de 5 requisições no intervalo de 1 minuto.
403	Not authenticated	Header Authorization ausente ou token inválido.

[← Venda Pix](#)[Tokenizar o Cartão →](#)

Tokenizar o Cartão

Produção

POST /api/payments/tokenize-card

Tokeniza um cartão, armazenando os dados de forma segura no **Card Vault**. Em caso de sucesso, a API retorna um `slugToken` (32 caracteres) usado depois em `POST /payments/pay` no campo `slugToken`.

A resposta é um **envelope** no formato `{ status_code, content }`. Para cartões armazenados no cofre, o campo análogo ao `slugToken` é `slugStoredCard`.

Autenticação (dupla)

Aceita `Authorization: Bearer <JWT>` **OU** o header `X-API-Key: <sua_api_key>` (integração server-to-server). Veja a página [API Keys](/docs/api-keys) (/docs/api-keys).

Exemplo de corpo

```
{
  "cardNumber": "4111111111111111",
  "cardExpirationDate": "122029",
  "cardHolderName": "JOAO SILVA",
  "cardSecurityCode": "123"
}
```

 Copiar

Headers

Authorization

Access token gerado em `POST /auth/token`. Alternativamente, use o header `X-API-Key`.

Bearer eyJhbGciOiJIUzI1NiIs...

Parâmetros do Body

cardNumber text required

Número do cartão (PAN).

4111111111111111

cardExpirationDate text required

Validade do cartão no formato `MMyyyy` (ex.: 122029).

cardHolderName text

Nome do portador conforme impresso no cartão.

cardSecurityCode text

Código de segurança do cartão (CVV).

[Testar requisição](#)[cURL](#)[Node.js](#)[Python](#)

```
curl -X POST "https://api.prod.pagmaxx.com/api/payments/tokenize-card" \
  -H "Content-Type: application/json" \
  -d '{}'
```

[Copiar](#)**Resposta:**

```
// Aguardando requisição...
```

Parâmetros do corpo

cardNumber string obrigatório	Número do cartão (PAN).
cardExpirationDate string obrigatório	Validade no formato MMyyyy (ex.: 122029).
cardHolderName string opcional	Nome do portador do cartão.
cardSecurityCode string opcional	Código de segurança do cartão (CVV).

Resposta

O corpo é um envelope { `status_code`, `content` }. O `content.slugToken` (32 caracteres) é o token a usar nas chamadas de venda em `/payments/pay`.

● 200 — Cartão tokenizado

```
{
  "status_code": 200,
  "content": {
    "slugToken": "9F3A2B1C4D5E6F7A8B9C0D1E2F3A4B5C",
    "maskedCard": "411111*****1111",
    "cardBrand": "VISA"
  }
}
```

[Copiar](#)

💡 Ao usar o `slugToken` na venda, não é necessário reenviar `cardNumber` nem `cardExpirationDate`. Para cartões no cofre, o campo análogo é `slugStoredCard`.

Erros

STATUS	DETAIL	QUANDO OCORRE
422	Data de validade do cartão inválida. Use o formato MMyyyy ...	cardExpirationDate fora do formato MMyyyy.
422	Estabelecimento ainda não habilitado para pagamento com cartão na adquirente (termo de adesão pendente). Contate o vendedor.	EC sem termo de adesão assinado na adquirente.
422	Cartão recusado pela adquirente: <msg>	Adquirente rejeitou os dados do cartão.
502	Erro na integracao com adquirente: <msg>	Falha de comunicação com a adquirente.

[← Obter Venda Pix](#)

[3DS Autenticação Setup →](#)

3DS Autenticação — Setup

Produção



POST /api/payments/3ds/authentication-setup

Primeira etapa do fluxo 3D Secure 2.x. Coleta os dados do dispositivo (device data collection) e prepara a transação de autenticação que será usada em /payments/3ds/authentication.

Envie **os dados do cartão** (PAN aberto) **ou** um slug de um cartão já salvo — nunca os dois. O campo ipAddress é injetado automaticamente pelo backend; você não precisa enviá-lo.

Autenticação

Este endpoint aceita **autenticação dupla**: envie Authorization: Bearer <JWT> **ou** o header X-API-Key. Para emitir e gerenciar chaves de API, veja [Chaves de API](#).

Exemplo de corpo (PAN aberto)

```
{
  "cardNumber": "4111111111111111",
  "cardExpirationDate": "1229",
  "cardHolderName": "JOAO SILVA"
}
```

 Copiar

Exemplo de corpo (slug)

```
{
  "slugToken": "a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6"
}
```

 Copiar

Headers

Authorization

Access token gerado em POST /auth/token. Alternativamente, autentique com o header X-API-Key.

Bearer eyJhbGciOiJIUzI1NiIs...

X-API-Key

Chave de API server-to-server. Use no lugar de Authorization quando integrar sem JWT.

pmx_live_...

Parâmetros do Body

cardNumber text

Número do cartão (PAN). Obrigatório se não usar slugToken/slugStoredCard.

cardExpirationDate text

Validade do cartão (MMyy ou MMyyyy). Obrigatória com PAN aberto.

cardHolderName text

Nome do portador. Obrigatório com PAN aberto.

slugToken text

Cartão tokenizado (32 chars) por /payments/tokenize-card. Substitui o PAN.

slugStoredCard text

Cartão armazenado no cofre (32 chars). Substitui o PAN.

[Testar requisição](#)[cURL](#)[Node.js](#)[Python](#)

```
curl -X POST "https://api.prod.pagmaxx.com/api/payments/3ds/authenticat
setup" \
\
-H "Content-Type: application/json" \
-d '{}'
```

[Copiar](#)

Resposta:



```
// Aguardando requisição...
```

Parâmetros do corpo

Use **um dos dois grupos** abaixo: os dados do cartão (PAN aberto) **ou** um slug. Eles são mutuamente exclusivos.

DADOS DO CARTÃO (PAN ABERTO)

cardNumber string opcional	PAN. Obrigatório se não usar slugToken/slugStoredCard.
cardExpirationDate string opcional	Validade no formato MMyy ou MMyyyy. Obrigatória com PAN aberto.
cardHolderName string opcional	Nome do portador. Obrigatório com PAN aberto.

OU SLUG (CARTÃO JÁ SALVO)

slugToken	Cartão tokenizado (32 chars). Substitui o PAN.
string opcional	

slugStoredCard	Cartão no cofre (32 chars). Substitui o PAN.
string opcional	

⚠️ Envie os dados do cartão **ou** um slug — nunca os dois ao mesmo tempo.

Resposta

A resposta vem em um **envelope** { status_code, content }. O objeto content é repassado integralmente pela adquirente e traz os dados necessários para a coleta de dados do dispositivo (device data collection), como referenceId, transactionId e a URL/parâmetros que você deve carregar para o setup.

● 200 — Setup criado

```
{
  "status_code": 200,
  "content": {
    "referenceId": "auth-ref-001",
    "transactionId": "a3c1f8e2-9f11-4c2e-b8a1-123456789abc",
    "deviceDataCollectionUrl":
"https://centinelapi.cardinalcommerce.com/V1/Cruise/Collect",
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  }
}
```

Copiar

💡 Os campos dentro de content são repassados pela adquirente e podem variar. Persista o referenceId e o transactionId para usar em /payments/3ds/authentication.

Erros

Quando a adquirente responde com status diferente de 200, o PagMaxx repassa esse status e o corpo com o erro da adquirente.

STATUS	DETAIL	QUANDO OCORRE
≠ 200	<erro repassado pela adquirente>	A adquirente recusou ou falhou na criação do setup; o status e o corpo são repassados.
403	Not authenticated	Header Authorization e X-API-Key ausentes ou inválidos.

[← Tokenizar o Cartão](#)

[3DS Autenticação →](#)

3DS Autenticação

Produção



POST

/api/payments/3ds/authentication

Segunda etapa do fluxo 3D Secure 2.x. Inicia a autenticação da transação. Se o emissor exigir o desafio (challenge), a resposta traz a URL do ACS para você exibir o desafio ao portador. Se o emissor liberar a transação sem desafio (frictionless), os dados de autenticação retornam direto.

Envie **os dados do cartão ou** um slug — mutuamente exclusivos. Os campos merchantUrl, ipAddress e threeDSRequestorChallengeIndicator são preenchidos/forçados pelo backend (o challenge é sempre "04").

Autenticação

Este endpoint aceita **autenticação dupla**: envie Authorization: Bearer <JWT> **ou** o header X-API-Key. Para emitir e gerenciar chaves de API, veja [Chaves de API](#).

Exemplo de corpo

```
{
  "type": "CREDIT",
  "transactionMode": "S",
  "acsWindowSize": "05",
  "deviceChannel": "Browser",
  "requestId": "c3f1e8b7-1b7d-4a9b-b9a5-9fbc9f3a1234",
  "referenceId": "auth-ref-001",
  "returnUrl": "https://seusite.com/3ds/return",
  "cardNumber": "4111111111111111",
  "cardExpirationDate": "1229",
  "cardHolderName": "JOAO SILVA",
  "orderInformation": {
    "amountDetails": { "currency": "BRL", "totalAmount": "100.00" },
    "billTo": {
      "firstName": "JOAO",
      "lastName": "SILVA",
      "address1": "Rua A, 123",
      "administrativeArea": "SP",
      "locality": "São Paulo",
      "country": "BR",
      "postalCode": "01001000",
      "email": "user@example.com",
      "phoneNumber": "+5511999999999"
    }
  },
  "buyerInformation": { "mobilePhone": "+5511999999999" },
  "deviceInformation": {
    "httpBrowserLanguage": "pt-BR",
    "httpBrowserColorDepth": "24",
    "httpBrowserScreenHeight": "1080",
    "httpBrowserScreenWidth": "1920",
    "httpAcceptContent": "text/html",
    "userAgentBrowserValue": "Mozilla/5.0"
  }
}
```

 Copiar

Headers

Authorization

Access token gerado em POST /auth/token. Alternativamente, autentique com o header X-API-Key.

Bearer eyJhbGciOiJIUzI1NiIs...

X-API-Key

Chave de API server-to-server. Use no lugar de Authorization quando integrar sem JWT.

pmx_live_...

Parâmetros do Body

type text **required**

CREDIT ou DEBIT.

CREDIT | DEBIT

transactionMode text **required**

Modo da transação: 1 caractere [M, R, S, P, T].

S

acsWindowSize text **required**

Tamanho da janela do ACS: 01 a 05.

05

deviceChannel text **required**

Browser, SDK ou 3RI.

Browser

requestId text **required**

Identificador da requisição de autenticação.

requestId

referenceId text **required**

referenceId retornado no authentication-setup.

referenceId

returnUrl text **required**

URL para onde o ACS retorna após o challenge.

https://seusite.com/3ds/return

cardNumber text

PAN. Obrigatório se não usar slug.

4111111111111111

cardExpirationDate text

Validade MMyy ou MMyyyy. Obrigatória com PAN aberto.

cardHolderName text

Nome do portador. Obrigatório com PAN aberto.

slugToken text

Cartão tokenizado (32 chars). Substitui o PAN.

slugStoredCard text

Cartão no cofre (32 chars). Substitui o PAN.

OrderInformation

amountDetails.currency text required

amountDetails.totalAmount text required

billTo.firstName text required

billTo.lastName text required

billTo.address1 text required

billTo.address2 text

Complemento (opcional).

billTo.administrativeArea text required

SP

billTo.locality text required

São Paulo

billTo.country text required

BR

billTo.postalCode text required

01001000

billTo.email email required

billTo.email

billTo.phoneNumber text required

billTo.phoneNumber

BuyerInformation

buyerInformation.mobilePhone text required

+5511999999999

DeviceInformation

deviceInformation.httpBrowserLanguage text required

pt-BR

deviceInformation.httpBrowserColorDepth text required

24

deviceInformation.httpBrowserScreenHeight text required

1080

deviceInformation.httpBrowserScreenWidth text required

deviceInformation.httpAcceptContent text required

deviceInformation.userAgentBrowserValue text required

deviceInformation.httpBrowserTimeDifference text

Diferença de fuso em minutos (opcional).

Testar requisição

cURL

Node.js

Python

```
curl -X POST "https://api.prod.pagmaxx.com/api/payments/3ds/authenticat  
\n  -H "Content-Type: application/json" \  
  -d '{}'
```

Copiar

Resposta:



```
// Aguardando requisição...
```

Parâmetros do corpo

PRINCIPAIS

type string obrigatório	CREDIT ou DEBIT.
transactionMode string obrigatório	1 caractere [M, R, S, P, T].
acsWindowSize string obrigatório	Tamanho da janela do ACS: 01 a 05.
deviceChannel string obrigatório	Browser, SDK ou 3RI.
requestId string obrigatório	Identificador da requisição de autenticação.
referenceId string obrigatório	referenceId retornado no authentication-setup.
returnUrl string obrigatório	URL para onde o ACS retorna após o challenge.

CARTÃO (OU SLUG) — MUTUAMENTE EXCLUSIVOS

cardNumber string opcional	PAN. Obrigatório se não usar slug.
cardExpirationDate string opcional	Validade MMyy ou MMyyyy. Obrigatória com PAN aberto.
cardHolderName string opcional	Nome do portador. Obrigatório com PAN aberto.
slugToken string opcional	Cartão tokenizado (32 chars). Substitui o PAN.
slugStoredCard string opcional	Cartão no cofre (32 chars). Substitui o PAN.

ORDERINFORMATION (OBJETO, OBRIGATÓRIO)

<code>orderInformation.amountDetails.currency</code> string obrigatório	Moeda (ex.: BRL).
<code>orderInformation.amountDetails.totalAmount</code> string obrigatório	Valor total da transação.
<code>orderInformation.billTo.firstName</code> string obrigatório	Primeiro nome do portador.
<code>orderInformation.billTo.lastName</code> string obrigatório	Sobrenome do portador.
<code>orderInformation.billTo.address1</code> string obrigatório	Endereço (rua e número).
<code>orderInformation.billTo.address2</code> string opcional	Complemento (opcional).
<code>orderInformation.billTo.administrativeArea</code> string obrigatório	Estado/UF.
<code>orderInformation.billTo.locality</code> string obrigatório	Cidade.
<code>orderInformation.billTo.country</code> string obrigatório	País (ex.: BR).
<code>orderInformation.billTo.postalCode</code> string obrigatório	CEP.
<code>orderInformation.billTo.email</code> string obrigatório	E-mail do portador.
<code>orderInformation.billTo.phoneNumber</code> string obrigatório	Telefone do portador.

BUYERINFORMATION E DEVICEINFORMATION

<code>buyerInformation.mobilePhone</code> string obrigatório	Celular do comprador.
<code>deviceInformation.httpBrowserLanguage</code> string obrigatório	Idioma do navegador (ex.: pt-BR).
<code>deviceInformation.httpBrowserColorDepth</code> string obrigatório	Profundidade de cor (ex.: 24).
<code>deviceInformation.httpBrowserScreenHeight</code> string obrigatório	Altura da tela (px).
<code>deviceInformation.httpBrowserScreenWidth</code> string obrigatório	Largura da tela (px).
<code>deviceInformation.httpAcceptContent</code> string obrigatório	Header Accept do navegador.
<code>deviceInformation.userAgentBrowserValue</code> string obrigatório	User-Agent do navegador.
<code>deviceInformation.httpBrowserTimeDifference</code> string opcional	Diferença de fuso em minutos (opcional).

Preenchidos pelo backend

Os campos `merchantUrl`, `ipAddress` e `threeDSRequestorChallengeIndicator` não precisam ser enviados — o PagMaxx os preenche/força. A PagMaxx **sempre** solicita o challenge (mandate "04").

Resposta

A resposta vem em um **envelope** { `status_code`, `content` }. Há dois cenários possíveis dentro de `content`:

● 200 — Challenge requerido

```
{
  "status_code": 200,
  "content": {
    "code": "WAITING_3DS_AUTHENTICATION",
    "authenticationId": "a3c1f8e2-9f11-4c2e-b8a1-123456789abc",
    "acsUrl": "https://acs.emissor.com/challenge",
    "acsTransactionId": "f8e2a3c1-4c2e-9f11-b8a1-abc123456789",
    "stepUpUrl": "https://acs.emissor.com/step-up"
  }
}
```

Copiar

● 200 — Frictionless (autenticação concluída sem desafio)

```
{
  "status_code": 200,
  "content": {
    "cavv": "AAABCZlhcQAAAABZlyFxAAAAAAA=",
    "eci": "05",
    "transStatus": "Y",
    "specificationVersion": "2.2.0",
    "directoryServerTransactionId": "f8e2a3c1-4c2e-9f11-b8a1-abc123456789"
  }
}
```

Copiar

 A PagMaxx sempre solicita o challenge (mandate). Se o emissor liberar frictionless, os dados de autenticação retornam direto e você já pode seguir para `/payments/pay`.

Erros

Quando a adquirente responde com status diferente de 200, o PagMaxx repassa esse status e o corpo com o erro da adquirente.

STATUS	DETAIL	QUANDO OCORRE
≠ 200	<erro repassado pela adquirente>	A adquirente recusou ou falhou na autenticação; o status e o corpo são repassados.
403	Not authenticated	Header Authorization e X-API-Key ausentes ou inválidos.

← 3DS Autenticação Setup

3DS Challenge Result →

3DS Challenge Result

POST /api/payments/3ds/challenge-result

Este endpoint é o **callback** (o `returnUrl`) que o ACS do banco aciona após o portador concluir o desafio 3D Secure. Normalmente **você não o chama diretamente** — quem o chama é o ACS do emissor.

Diferente dos outros endpoints do fluxo, ele **não recebe nem responde JSON**: os dados chegam em `application/x-www-form-urlencoded` e a resposta é uma **página HTML** que notifica a janela pai (a sua) via `window.parent.postMessage`.

Como integrar

Carregue o desafio (ACS) num `iframe` e escute `window.addEventListener('message', ...)`. Em `3DS_CHALLENGE_COMPLETE`, use o `threeDsData` recebido no campo `threeDsData` de `/payments/pay` (ou `/payments/pay-secure`).

Como funciona o fluxo

No passo de `/payments/3ds/authentication`, quando o emissor exige challenge, você envia um `returnUrl` apontando para este endpoint. A sequência é:

1. Você renderiza o desafio do ACS (geralmente dentro de um `iframe`) para o portador.
2. O portador conclui o desafio no ACS do banco.
3. O ACS faz um **POST em form-urlencoded** neste `returnUrl` (`/3ds/challenge-result`).
4. O endpoint resolve o resultado e devolve um **HTML** que chama `window.parent.postMessage(...)` para notificar a sua janela.
5. Sua página recebe a mensagem; em caso de sucesso, usa o `threeDsData` na chamada final de `/payments/pay` (ou `/payments/pay-secure`).

Campos recebidos (`application/x-www-form-urlencoded`)

O ACS envia os campos abaixo no corpo da requisição:

TransactionId string obrigatório	ID da transação de autenticação.
MD string obrigatório	O requestId enviado na chamada de /payments/3ds/authentication. É por ele que a autenticação registrada é localizada.
Response string opcional	Resposta do ACS (opcional).

Resposta — HTML com postMessage

A resposta é uma página HTML. Há dois retornos possíveis: sucesso e falha. Em ambos, a página chama `window.parent.postMessage(payload, "*")` para a sua janela.

● Sucesso — type "3DS_CHALLENGE_COMPLETE"

```
<!DOCTYPE html><html><body><script>
window.parent.postMessage({ type: "3DS_CHALLENGE_COMPLETE", threeDsData: { "cavv":
"...", "secureVersion": "2.2.0", "directoryServerTransactionId": "..." } }, "*");
</script></body></html>
```

 Copiar

● Falha — type "3DS_CHALLENGE_FAILED"

```
<!DOCTYPE html><html><body><script>
window.parent.postMessage({ type: "3DS_CHALLENGE_FAILED", error: "Autenticacao 3DS
rejeitada", reason: "O banco emissor recusou a autenticacao deste cartao. Fale com o
banco ou pague com outro cartao.", result: "FAILED" }, "*");
</script></body></html>
```

 Copiar

 No payload de sucesso, o objeto `threeDsData` já vem no formato esperado pelo campo `threeDsData` de `/payments/pay` — basta repassá-lo. No payload de falha, `reason` traz o motivo já traduzido para português, pronto para exibir ao pagador; `error` e `result` seguem inalterados.

Erros

STATUS	DETAIL	QUANDO OCORRE
404	Transaction not found - <MD>	O MD/requestId não corresponde a nenhuma autenticação registrada.

Simular Taxa

Produção

POST /api/payments/simulate-fee

Simula as taxas aplicadas a uma transação antes da venda, sem executá-la. A partir do `cardNumber` a adquirente identifica a bandeira e a tabela de taxas e calcula, por número de parcelas, a **taxa (MDR)** e o **valor líquido**.

A resposta é um **envelope** no formato { `status_code`, `content` }, onde `content` é o retorno do simulador de taxas **repassado pela adquirente**.

Autenticação

Este endpoint usa **Bearer JWT** (Authorization), gerado em `POST /auth/token`.

Rate limit

Este endpoint aceita até **100 requisições por minuto**.

Exemplo de corpo

```
{
  "amount": "100.00",
  "cardNumber": "4111111111111111"
}
```

 Copiar

Headers

Authorization

Access token gerado em `POST /auth/token`.

Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLT...

Parâmetros do Body

amount text required

Valor base da simulação.

100.00

cardNumber text required

Cartão usado para identificar a bandeira e a tabela de taxas.

4111111111111111

Testar requisição

cURL

Node.js

Python

```
curl -X POST "https://api.prod.pagmaxx.com/api/payments/simulate-fee" \
  -H "Content-Type: application/json" \
  -d '{}'
```

Copiar

Resposta:

● ---

```
// Aguardando requisição...
```

Parâmetros do corpo

amount
string **obrigatório**

Valor base da simulação (ex.: 100.00).

cardNumber
string **obrigatório**

Cartão usado para identificar a bandeira e a tabela de taxas.

Resposta

O corpo é um envelope { status_code, content }. O campo content é o retorno do simulador de taxas **repassado pela adquirente** e pode variar por bandeira e cenário — o exemplo abaixo é ilustrativo, com taxa e valor líquido por número de parcelas.

● 200 — Simulação

```
{
  "status_code": 200,
  "content": {
    "amount": 100.00,
    "cardBrand": "VISA",
    "installmentOptions": [
      { "installments": 1, "feePercent": 3.49, "feeAmount": 3.49, "netAmount": 96.51 },
      { "installments": 2, "feePercent": 5.19, "feeAmount": 5.19, "netAmount": 94.81 },
      { "installments": 3, "feePercent": 6.89, "feeAmount": 6.89, "netAmount": 93.11 }
    ]
  }
}
```

[Copiar](#)

💡 Como content é repassado pela adquirente, trate-o de forma defensiva: os nomes e a estrutura dos campos podem variar conforme a tabela de taxas do estabelecimento.

[← 3DS Challenge Result](#)[Assinaturas →](#)

Assinaturas (Recorrência)

Produção

POST /api/subscriptions/

Crie cobranças recorrentes no cartão (MIT) ou via Pix. Você cria a assinatura pela API e recebe uma **URL pública de autorização** (`public_url`) — o pagador acessa essa página hospedada pela PagMaxx para autorizar o pagamento. As cobranças seguintes são processadas automaticamente.

Como funciona (modelo hosted)

1. Você cria a assinatura (POST /subscriptions/) e recebe `public_url`.
2. O pagador acessa a `public_url` e autoriza: no **cartão**, informa o cartão + 3DS (cobrança imediata do 1º ciclo + mandato MIT); no **Pix**, paga o QR do ciclo atual.
3. A PagMaxx executa os ciclos seguintes automaticamente (cartão: MIT; Pix: gera novo QR e notifica o pagador por e-mail).

Exemplo de corpo

```
{
  "payer_name": "João Silva",
  "payer_email": "pagador@email.com",
  "amount": 59.90,
  "method": "CREDIT",
  "interval": "MONTHLY",
  "interval_count": 1,
  "cycles_total": 12,
  "title": "Plano Premium",
  "external_reference": "PEDIDO-9987",
  "require_3ds": true,
  "pass_fee": false,
  "fee_brand": "VISA"
}
```

 Copiar

Exemplo de corpo — com split percentual

```
{
  "payer_name": "João Silva",
  "payer_email": "pagador@email.com",
  "amount": 200.00,
  "method": "CREDIT",
  "interval": "MONTHLY",
  "cycles_total": 12,
  "title": "Plano Premium",
  "split_config": {
    "split_type": "PERCENTUAL",
    "items": [
      { "partner_id": "5ca18d4a-4a92-4b45-8c0a-9bf6f490cb85", "value": 30 }
    ]
  }
}
```

[Copiar](#)

Headers

Authorization

Access token gerado em POST /auth/token.

Bearer eyJhbG...

Parâmetros do Body

payer_name text **required**

Nome do pagador (2 a 255 caracteres).

João Silva

payer_email email **required**

E-mail do pagador (recebe cobranças/notificações).

pagador@email.com

amount text **required**

Valor de cada cobrança (> 0).

59.90

method text

CREDIT (cartão/MIT) ou PIX. Padrão: CREDIT.

CREDIT

interval text

WEEKLY | MONTHLY | QUARTERLY | SEMIANNUAL | ANNUAL. Padrão: MONTHLY.

MONTHLY

interval_count number

Multiplicador do intervalo (1 a 12). Padrão: 1.

1

cycles_total number

Número total de cobranças (1 a 120). Vazio = assinatura infinita até cancelar.

12

payer_document_id text

CPF (11) ou CNPJ (14) do pagador. Opcional.

12345678901

title text

Título exibido ao pagador. Opcional.

Plano Premium

description text

Descrição da assinatura. Opcional.

description

require_3ds text

Exigir 3DS na autorização do cartão. Padrão: true.

true

pass_fee text

true = repassar a taxa ao assinante. Com ele, amount passa a ser o LÍQUIDO desejado e o servidor calcula o valor cobrado. Padrão: false.

false

fee_brand text

Bandeira de referência da taxa (VISA, MASTERCARD, ELO, AMEX, HIPERCARD, CABAL). Ausente = pior taxa entre as bandeiras.

Split_config**split_config.split_type** text

PERCENTUAL ou ABSOLUTE. Obrigatório quando há split. Somente com method=CREDIT.

split_config.items[0].partner_id text

ID do parceiro que recebe a parte. Obtido em /api/partners.

split_config.items[0].value text

Percentual (ex.: 30 = 30%) ou valor absoluto em reais.

[Testar requisição](#)[cURL](#)[Node.js](#)[Python](#)

```
curl -X POST "https://api.prod.pagmaxx.com/api/subscriptions/" \
  -H "Content-Type: application/json" \
  -d '{}'
```

[Copiar](#)

Resposta:



```
// Aguardando requisição...
```

Parâmetros do corpo

payer_name string obrigatório	Nome do pagador (2–255).
payer_email string obrigatório	E-mail do pagador.
amount number obrigatório	Valor de cada cobrança (> 0).
method string opcional	CREDIT (cartão/MIT) ou PIX. Padrão CREDIT.
interval string opcional	WEEKLY MONTHLY QUARTERLY SEMIANNUAL ANNUAL. Padrão MONTHLY.
interval_count number opcional	Multiplicador do intervalo (1–12). Padrão 1.
cycles_total number opcional	Total de cobranças (1–120). Ausente = infinita.
payer_document_id string opcional	CPF (11) ou CNPJ (14). Opcional.
title string opcional	Título exibido ao pagador. Opcional.
description string opcional	Descrição. Opcional.
external_reference string opcional	Sua referência para esta assinatura — id do pedido, contrato ou cliente no seu sistema (até 255 caracteres). Não interpretamos o conteúdo: ele volta em todo webhook de cobrança desta assinatura, no campo <code>payment.subscription_external_reference</code> . Opcional.
require_3ds boolean opcional	Exigir 3DS na autorização do cartão. Padrão true.
pass_fee boolean opcional	Repassa a taxa ao assinante. Com true, amount vira o LÍQUIDO desejado e o servidor devolve em amount o valor

	efetivamente cobrado (net_amount guarda o líquido). Padrão false.
fee_brand string opcional	Bandeira de referência do cálculo: VISA, MASTERCARD, ELO, AMEX, HIPERCARD ou CABAL. A taxa varia por bandeira. Ausente = usa a pior taxa do seu catálogo (conservador). Ignorado quando method=PIX, que tem taxa fixa.
split_config object opcional	Divisão automática do valor entre parceiros, aplicada em TODOS os ciclos da assinatura. Somente no crédito. Opcional.

split_config (objeto)

O split divide automaticamente cada cobrança da assinatura entre você e seus parceiros — não só a primeira: o rateio é gravado na criação e reenviado em **todos os ciclos seguintes**, inclusive nas cobranças automáticas no cartão.

split_config.split_type string obrigatório	PERCENTUAL (value em %) ou ABSOLUTE (value em reais).
split_config.items[] array obrigatório	De 1 a 20 participantes. Não pode repetir o mesmo parceiro.
split_config.items[].partner_id string obrigatório	ID do parceiro, obtido em GET /api/partners. Precisa estar com credenciamento ativo.
split_config.items[].value number obrigatório	Parte do parceiro. Em PERCENTUAL, 30 significa 30%. Em ABSOLUTE, é o valor em reais.
split_config.source_split_rule_id string opcional	ID de uma regra de split salva, para registrar a origem da configuração.

Regras do split em assinatura

- Exige method: "CREDIT". Enviar split_config com method: "PIX" é recusado com 422.
- A soma das partes é validada contra o amount da assinatura e não pode atingir o total.
- Convive com pass_fee: a taxa é cobrada de cada participante sobre a própria fatia, então o rateio incide sobre o valor já com repasse.

- Depois de criada, `split_config` **não pode ser alterado**. Em assinatura com `split ABSOLUTE`, alterar o `amount` é bloqueado com 409 — o rateio precisa ser feito antes.

Quando há `split`, a resposta da criação traz o campo `split` preenchido com o snapshot dos participantes (vem `null` quando não há):

Campo `split` na resposta

```
"split": {
  "split_type": "PERCENTUAL",
  "source_split_rule_id": null,
  "items": [
    {
      "partner_id": "5ca18d4a-4a92-4b45-8c0a-9bf6f490cb85",
      "partner_name": "Parceiro Exemplo",
      "partner_document_id": "12345678000199",
      "value": 30.0
    }
  ]
}
```

 Copiar

Taxa e repasse (`pass_fee` + `fee_brand`)

A taxa de crédito muda conforme a bandeira. Com `pass_fee: true`, o valor enviado em `amount` passa a ser o **líquido que você quer receber**: o servidor calcula o bruto cobrado do assinante usando a taxa de `fee_brand` e devolve esse bruto no campo `amount` da resposta (o líquido fica em `net_amount`). O valor é **congelado na criação** e vale para todos os ciclos da assinatura.

Sem `fee_brand`, o cálculo usa a **pior taxa** entre as suas bandeiras — conservador, você nunca recebe menos do que pediu. Informando a bandeira, o valor sai pela taxa dela. O assinante continua livre para pagar com qualquer cartão: se a bandeira real for diferente da informada, o valor cobrado não muda (já está congelado), mas o líquido creditado segue a taxa da bandeira efetivamente usada.

Em `method: "PIX"` a taxa é um valor fixo em reais por cobrança, definido no cadastro do estabelecimento (não é percentual), e `fee_brand` é ignorado.

Resposta

Retorna 201 com a assinatura criada. Envie a `public_url` ao pagador para que ele autorize. Enquanto não autorizada, a assinatura no cartão fica com status `pending_authorization`; no Pix, já nasce `active` com o 1º ciclo agendado.

```
{
  "id": "9b1c...",
  "public_id": "3f2a7c10-8e44-4b9a-9c12-7d5e6f8a1b2c",
  "public_url": "https://portal.pagmaxx.com/assinatura/3f2a7c10-8e44-4b9a-9c12-7d5e6f8a1b2c",
  "payer_name": "João Silva",
  "payer_email": "pagador@email.com",
  "method": "CREDIT",
  "amount": 59.90,
  "pass_fee": false,
  "net_amount": null,
  "fee_brand": "VISA",
  "currency": "BRL",
  "interval": "MONTHLY",
  "interval_count": 1,
  "cycles_total": 12,
  "cycles_done": 0,
  "external_reference": "PEDIDO-9987",
  "status": "pending_authorization",
  "next_charge_at": null,
  "require_3ds": true,
  "card_brand": null,
  "card_last4": null,
  "split": null,
  "created_at": "2026-06-17T12:00:00Z"
}
```

 Copiar



Status da assinatura

pending_authorization → aguardando o pagador autorizar (cartão) · active → vigente · past_due → cobrança em atraso · paused → pausada · canceled → cancelada.

Página pública de autorização

A `public_url` aponta para os endpoints públicos (sem autenticação, escopados pela assinatura) que a página hospedada consome:

GET /subscriptions/public/{public_id} público opcional	Dados da assinatura para exibir ao pagador.
POST /subscriptions/public/{public_id}/tokenize público opcional	Tokeniza o cartão informado pelo pagador.
POST /subscriptions/public/{public_id}/3ds/setup público opcional	Setup do 3DS.
POST /subscriptions/public/{public_id}/3ds/authenticate público opcional	Autenticação 3DS.
POST /subscriptions/public/{public_id}/authorize público opcional	Autoriza o cartão (1ª cobrança + mandato MIT).
POST /subscriptions/public/{public_id}/pix público opcional	Gera o QR Pix do ciclo em aberto.

❗ Na maioria das integrações você só precisa criar a assinatura e redirecionar o pagador para a `public_url` — a página hospedada cuida do restante.

❗ **Não fixe o domínio da `public_url` no seu código.** Ele é calculado na resposta: se a sua conta usa domínio próprio, a URL já sai nele. Use sempre a `public_url` que veio na resposta — links antigos continuam funcionando.

❗ **whitelabel — marca do estabelecimento**

A resposta de GET /subscriptions/public/{public_id} traz o campo opcional `whitelabel`: um objeto com `name`, `logo_url`, `favicon_url`, `color_primary`, `color_primary_contrast`, `support_email`, `footer_text` e `is_default`, usado para renderizar a página hospedada com a marca do estabelecimento dono da assinatura. Vem `null` quando o estabelecimento não usa `whitelabel` — a página segue idêntica à de hoje.

Erros

STATUS	DETAIL	QUANDO OCORRE
422	Erro de validação (ex.: amount ≤ 0, documento inválido)	Campos do corpo inválidos.
403	Not authenticated	Sem Authorization válido.
422	Split disponivel apenas para assinatura no Cartao de Credito.	split_config enviado com method diferente de CREDIT.
422	Split exige ao menos 1 participante.	split_config.items vazio.
422	Maximo de 20 participantes por split.	Mais de 20 itens em split_config.items.
422	Parceiro nao esta com credenciamento ativo na PagMaxx	Algum parceiro do split ainda não concluiu o credenciamento.
422	Soma dos participantes ...	A soma do split atinge ou ultrapassa o valor da cobrança.
422	Regra de split de origem nao encontrada.	source_split_rule_id inexistente ou de outro estabelecimento.

[← Simular Taxa](#)

[Gerenciar Assinaturas →](#)

Gerenciar Assinaturas

Produção

GET /api/subscriptions/

Consulte e controle o ciclo de vida das assinaturas do seu estabelecimento: listar, detalhar, ver o histórico de cobranças, editar, pausar, retomar, cancelar e reenviar o Pix do ciclo em aberto. Todos os endpoints usam Authorization: Bearer <JWT>.

Use o testador abaixo para **listar** as assinaturas. Opcionalmente filtre por status (query param).

Headers

Authorization

Access token gerado em POST /auth/token.

Bearer eyJhbG...

Query Parâmetros

status text

Filtra por status: pending_authorization | active | past_due | paused | canceled. Opcional.

active

Testar requisição

cURL

Node.js

Python

```
curl -X GET "https://api.prod.pagmaxx.com/api/subscriptions/" \
-H "Content-Type: application/json" \
-d '{}'
```

Copiar

Resposta:

```
// Aguardando requisição...
```

Listar assinaturas — GET /api/subscriptions/

Retorna o array de assinaturas do estabelecimento. Aceita o query param opcional status.

● HTTP 200

```
[
  {
    "public_id": "3f2a7c10-...",
    "payer_name": "João Silva",
    "method": "CREDIT",
    "amount": 59.90,
    "interval": "MONTHLY",
    "status": "active",
    "cycles_done": 3,
    "cycles_total": 12,
    "next_charge_at": "2026-07-17T00:00:00Z",
    "card_brand": "VISA",
    "card_last4": "1111"
  }
]
```

 Copiar

Detalhar — GET /api/subscriptions/{public_id}

Retorna uma assinatura específica (mesmo objeto do retorno de criação).

```
curl https://api.prod.pagmaxx.com/api/subscriptions/3f2a7c10-... \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

 Copiar

Histórico de cobranças — GET

/api/subscriptions/{public_id}/charges

Lista os ciclos de cobrança, em ordem crescente.

● HTTP 200

```
[
  {
    "cycle_number": 1,
    "due_date": "2026-06-17T00:00:00Z",
    "amount": 59.90,
    "status": "paid",
    "attempt_count": 1,
    "processed_at": "2026-06-17T12:00:05Z",
    "last_error": null,
    "payment_id": "70b0d9f7-3c58-4e21-9f0d-6a1b8c2e5d34"
  },
  {
    "cycle_number": 2,
    "due_date": "2026-07-17T00:00:00Z",
    "amount": 59.90,
    "status": "retrying",
    "attempt_count": 2,
    "processed_at": null,
    "last_error": "Saldo ou limite insuficiente. [AUTHORIZER_REJECTED] protocolo
5B62AB597B8B45169C94272745791BD4",
    "payment_id": "c4a91f52-77d0-4b18-8e3a-2f6c1d09b4e7"
  },
  {
    "cycle_number": 3,
    "due_date": "2026-08-17T00:00:00Z",
    "amount": 59.90,
    "status": "scheduled",
    "attempt_count": 0,
    "processed_at": null,
    "last_error": null,
    "payment_id": null
  }
]
```

 Copiar

SUBSCRIPTIONCHARGE

cycle_number number opcional	Número do ciclo.
due_date datetime opcional	Vencimento do ciclo.
amount number opcional	Valor do ciclo.
status string opcional	scheduled paid awaiting_pix retrying failed.
attempt_count number opcional	Tentativas de cobrança.
processed_at datetime opcional	Quando foi processado.
last_error string opcional	Motivo da última falha, em pt-BR, com o código da adquirente e o protocolo da transação. Null quando não houve falha.
payment_id string opcional	Id da venda gerada por este ciclo — o mesmo payment.id que chega no webhook. Use para reconciliar a fatura com o evento recebido. Null enquanto o ciclo não gerou cobrança (scheduled) ou quando a tentativa falhou antes de criar a venda.

Falha da adquirente não cancela a assinatura

Uma cobrança recusada pelo **banco emissor** segue o fluxo de inadimplência: novas tentativas em D+1, D+3 e D+5 e, se todas falharem, a assinatura é cancelada. Já uma **falha de processamento da adquirente** — quando a cobrança sequer chega ao emissor — não é inadimplência: o ciclo passa a tentar **uma vez por dia** e a assinatura **nunca** é cancelada por esse motivo. Após 30 dias sem sucesso o ciclo para de tentar e a assinatura fica em `past_due`, aguardando sua decisão. Nos dois casos o motivo fica em `last_error`.

Editar — PATCH /api/subscriptions/{public_id}

Atualiza cadastro, valor e periodicidade de uma assinatura ativa. Envie no corpo **somente os campos alterados** — campos omitidos mantêm o valor atual, e enviar `null` num campo opcional (ex.: `description`) o limpa. A edição vale só **daqui para frente**: o ciclo já processado ou em processamento (o próximo a vencer) não é retroativamente alterado, exceto quando ainda está `scheduled` — nesse caso o valor e o vencimento do ciclo em aberto acompanham a edição.

PATCH

/subscriptions/{public_id}

JWT (dono do EC) opcional

Edita a assinatura. Só o perfil Completo do estabelecimento pode editar (mesma trava do cancelamento).

CORPO — CAMPOS ACEITOS (TODOS OPCIONAIS)

payer_name string opcional	Nome do pagador.
payer_email string opcional	E-mail do pagador.
payer_document_id string opcional	CPF/CNPJ, só dígitos.
title string opcional	Identificador do plano.
description string opcional	Descrição visível ao pagador.
amount string opcional	Valor por cobrança (ex.: "150.00"). Mesma semântica da criação: se a assinatura tem pass_fee=true, este valor é o LÍQUIDO que o EC quer receber — o servidor recalcula o bruto cobrado do assinante. Bloqueado com 409 quando há split de valor fixo (ABSOLUTE).
interval string opcional	WEEKLY MONTHLY QUARTERLY SEMIANNUAL ANNUAL.
interval_count number opcional	Multiplicador do intervalo (1 a 12).
anchor_day number opcional	Dia-âncora do mês (1 a 31); ignorado em WEEKLY.
cycles_total number null opcional	Total de cobranças (1 a 120) ou null para assinatura infinita. Deve ser maior que o número de ciclos já cobrados; para encerrar, use o cancelamento.

```
curl -X PATCH https://api.prod.pagmaxx.com/api/subscriptions/3f2a7c10-... \
-H "Authorization: Bearer <ACCESS_TOKEN>" \
-H "Content-Type: application/json" \
-d '{ "amount": "79.90", "cycles_total": 12 }'
```

[Copiar](#)

● HTTP 200

```
{
  "public_id": "3f2a7c10-...",
  "amount": 79.90,
  "pass_fee": false,
  "net_amount": null,
  "interval": "MONTHLY",
  "cycles_total": 12,
  "cycles_done": 3,
  "status": "active",
  "next_charge_at": "2026-09-04T00:00:00Z"
}
```

 Copiar

STATUS	DETAIL	QUANDO OCORRE
409	Assinatura encerrada nao pode ser editada.	Status canceled ou completed.
409	Assinatura com split de valor fixo: o rateio precisa ser feito antes de alterar o valor.	amount enviado numa assinatura com split ABSOLUTE.
409	Total de cobranças deve ser maior que os N ciclo(s) já cobrado(s).	cycles_total menor ou igual ao número de ciclos já cobrados.
409	Nenhuma alteração a aplicar.	Corpo enviado sem diferença em relação ao estado atual.
422	Extra inputs are not permitted	Corpo com campo fora da lista aceita (ex.: method, split_config).
403	Not authenticated	Perfil Operador (Vendas) tentando editar — exige perfil Completo.

Campos imutáveis

method, kind, require_3ds, fee_brand, split_config e os dados do cartão não podem ser alterados por este endpoint — o schema usa extra="forbid" e devolve 422 se algum desses campos for enviado. Para trocar a forma de pagamento ou o rateio, cancele e crie uma nova assinatura.

Pausar / Retomar / Cancelar

POST /subscriptions/{public_id}/pause JWT opcional	Pausa a assinatura (suspende cobranças).
POST /subscriptions/{public_id}/resume JWT opcional	Retoma uma assinatura pausada.
POST /subscriptions/{public_id}/cancel JWT (dono do EC) opcional	Cancela definitivamente. Corpo opcional: { "reason": "..."}.

```
curl -X POST https://api.prod.pagmaxx.com/api/subscriptions/3f2a7c10-.../cancel \
-H "Authorization: Bearer <ACCESS_TOKEN>" \
-H "Content-Type: application/json" \
-d '{ "reason": "Solicitado pelo cliente" }'
```

[Copiar](#)

⚠ Permissão

Cancelar exige o perfil dono do estabelecimento (Completo). Pausar e retomar podem ser feitos por operadores. As três operações retornam a assinatura atualizada e devolvem 409 quando a transição não é permitida pelo estado atual.

Reenviar Pix — POST /api/subscriptions/{public_id}/resend-pix

Reenvia, por e-mail, o Pix do ciclo em aberto (apenas para assinaturas Pix ativas).

STATUS	DETAIL	QUANDO OCORRE
400	Reenvio de PIX so para assinaturas PIX.	A assinatura é de cartão.
409	Assinatura nao esta ativa.	Assinatura pausada/cancelada.
400	Assinatura sem e-mail do pagador.	Pagador sem e-mail cadastrado.
404	Nenhuma cobrança PIX em aberto para reenviar.	Não há ciclo Pix pendente.
502	Nao foi possivel enviar o e-mail. Tente novamente.	Falha no envio do e-mail.

Trocar cartao — POST /api/subscriptions/{public_id}/card-update

Abre uma janela de troca de cartao para uma assinatura de **cartao de credito**. A assinatura em si nao muda nesse momento: o portador recebe, no mesmo link publico da assinatura, um formulario para cadastrar o novo cartao. Exige o mesmo perfil dono do estabelecimento (Completo) que o cancelamento. Quando ha actingEcId (backoffice agindo em nome do EC), use POST /internal/admin/subscriptions/{actingEcId}/{public_id}/card-update — mesma ramificação já usada por editar/pausar/cancelar.

```
curl -X POST https://api.prod.pagmaxx.com/api/subscriptions/3f2a7c10-.../card-update -H "Authorization: Bearer <ACCESS_TOKEN>"
```

 Copiar

● HTTP 200

```
{
  "public_id": "3f2a7c10-...",
  "method": "CREDIT",
  "status": "active",
  "card_brand": "VISA",
  "card_last4": "1111",
  "card_update_requested_at": "2026-08-04T14:32:10Z"
}
```

 Copiar

STATUS	DETAIL	QUANDO OCORRE
409	<motivo da recusa>	Assinatura Pix (troca so vale para cartao), status fora de active/past_due, ou ja existe uma troca de cartao pendente.
404	Assinatura nao encontrada neste estabelecimento.	public_id inexistente ou de outro EC.
403	Not authenticated	Perfil Operador (Vendas) tentando abrir a troca — exige perfil Completo.

Cancelar troca de cartao — DELETE

/api/subscriptions/{public_id}/card-update

Cancela a janela de troca de cartao aberta pelo endpoint acima. A partir da resposta, o link publico volta a não aceitar cartão (como se a troca nunca tivesse sido aberta). Mesma trava de perfil e mesma ramificação actingEcId do endpoint acima.

```
curl -X DELETE https://api.prod.pagmaxx.com/api/subscriptions/3f2a7c10-.../card-update \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

● HTTP 200

```
{
  "public_id": "3f2a7c10-...",
  "card_update_requested_at": null
}
```

STATUS	DETAIL	QUANDO OCORRE
409	Nao ha troca de cartao pendente para cancelar.	Nenhuma solicitação de troca em aberto no momento da chamada.
404	Assinatura nao encontrada neste estabelecimento.	public_id inexistente ou de outro EC.

Confirmar novo cartao (link público) — POST

/api/subscriptions/public/{public_id}/update-card

Endpoint público, chamado pelo portador no mesmo link de autorização da assinatura (GET /api/subscriptions/public/{public_id}). Só aceita cartão

enquanto **card_update_pending** estiver **true** nesse GET — ou seja, só funciona enquanto a troca estiver pendente (aberta pelo card-update acima e ainda não confirmada nem cancelada). O corpo é **idêntico** ao já usado em /authorize: slug_stored_card/slug_token, card_security_code, card_holder_name, card_brand, three_ds_data e payer.

GET /API/SUBSCRIPTIONS/PUBLIC/{PUBLIC_ID} — CAMPOS NOVOS

card_update_pending boolean opcional	true enquanto houver uma troca de cartão pendente para essa assinatura. Controla se update-card aceita a chamada.
card_brand string null opcional	Bandeira do cartão atualmente cadastrado (o que será substituído).
card_last4 string null opcional	Últimos 4 dígitos do cartão atualmente cadastrado.
open_charge_amount string null opcional	Valor da cobrança já prevista (ciclo em aberto) que será paga com o novo cartão. Null quando não há ciclo em aberto no momento.
open_charge_due_date datetime null opcional	Vencimento da cobrança em aberto.
whitelabel object null opcional	Marca do estabelecimento dono da assinatura, para renderizar a página com o visual dele (name, logo_url, favicon_url, color_primary, color_primary_contrast, support_email, footer_text, is_default). Null quando o estabelecimento não usa whitelabel.

```
curl -X POST
https://api.prod.pagmaxx.com/api/subscriptions/public/3f2a7c10-.../update-card \
-H "Content-Type: application/json" \
-d '{
  "slug_stored_card": "abc123",
  "card_security_code": "123",
  "card_holder_name": "Joao Silva",
  "card_brand": "VISA",
  "payer": { "name": "Joao Silva", "document": "12345678900", "email":
"joao@email.com" }
}'
```

 Copiar

● HTTP 200

```
{
  "status": "updated",
  "subscription_status": "active",
  "payment_id": "pay_9f2a...",
  "card_brand": "VISA",
  "card_last4": "4444",
  "next_charge_at": "2026-09-04T00:00:00Z"
}
```

Copiar

STATUS	DETAIL	QUANDO OCORRE
4xx	Pagamento recusado / autenticação 3DS reprovada.	Resposta no formato { "status": "failed", "detail": "<motivo>" } — mesmo formato de erro do /authorize.
409	Nao ha troca de cartao pendente para esta assinatura.	card_update_pending já era false (troca já confirmada, cancelada, ou nunca aberta) no momento da chamada.
404	Not Found	public_id inexistente.

Cobra o ciclo em aberto, sem mexer no calendário

Ao confirmar o novo cartão, a plataforma cobra a cobrança **já prevista** (o ciclo em aberto) nesse novo cartão — **sem cobrança extra** e **sem alterar** as datas da assinatura (próxima cobrança, ciclos etc.). Trocar o cartão não antecipa nem posterga nenhum vencimento.

Erros comuns

STATUS	DETAIL	QUANDO OCORRE
404	Assinatura nao encontrada neste estabelecimento.	public_id inexistente ou de outro EC.
409	<motivo da transição inválida>	Pause/resume/cancel/edit não permitido pelo estado atual.
403	Not authenticated	Sem Authorization válido.

← Assinaturas

Webhooks →

Webhooks

A PagMaxx envia uma requisição POST para a URL que você cadastrar, sempre que houver uma mudança relevante em uma **venda** ou em uma **liquidação**. É a forma recomendada de manter seu sistema atualizado sem precisar ficar consultando a API periodicamente.

Como configurar

No portal, em **Configurações** → **Webhooks**, informe a URL que vai receber os eventos, gere o **segredo de assinatura** e, se quiser, dispare um evento de teste (test.ping) para validar a integração.

 **A URL precisa ser HTTPS e pública**

Endereços internos ou localhost são recusados no cadastro. A URL precisa usar **HTTPS** e ser acessível pela internet.

A mesma tela mostra a lista das **últimas entregas**, com o motivo de cada falha, e um botão para **reenviar** manualmente uma entrega específica.

Eventos

O evento disparado vem no campo `type` do corpo da requisição:

TYPE	QUANDO OCORRE
<code>payment.pending</code>	Venda criada e aguardando confirmação de pagamento.
<code>payment.authorized</code>	Pagamento confirmado, em cartão de crédito, cartão de débito ou PIX. É o evento que indica venda paga — no portal, ela passa a aparecer com o status "Aprovado".
<code>payment.approved</code>	Não é disparado atualmente. Corresponde à captura em fluxo de duas etapas; a plataforma confirma a venda em etapa única.
<code>payment.failed</code>	Pagamento recusado ou falhou.
<code>payment.expired</code>	Venda expirou sem confirmação de pagamento.
<code>payment.refunded</code>	Pagamento estornado ou cancelado, no todo ou em parte.
<code>payment.chargeback</code>	Chargeback recebido para o pagamento.
<code>payout.scheduled</code>	Liquidação agendada. Só é enviado se a opção "Receber eventos de liquidação" estiver ativada em Configurações → Webhooks.
<code>payout.paid</code>	Liquidação paga. Só é enviado se a opção "Receber eventos de liquidação" estiver ativada em Configurações → Webhooks.
<code>payout.canceled</code>	Liquidação cancelada. Só é enviado se a opção "Receber eventos de liquidação" estiver ativada em Configurações → Webhooks.
<code>payout.updated</code>	Dados ou status da liquidação foram atualizados. Só é enviado se a opção "Receber eventos de liquidação" estiver ativada em Configurações → Webhooks.
<code>test.ping</code>	Evento de teste, disparado pelo botão "Enviar teste" no portal.

Qual evento confirma o pagamento

A confirmação de pagamento é sinalizada pelo evento `payment.authorized` — em cartão de crédito, cartão de débito e PIX. Ao recebê-lo, a venda está paga e o produto ou serviço pode ser liberado ao cliente; no portal, ela aparece com o status **Aprovado**.

O evento `payment.approved` **não é disparado atualmente**: corresponde à captura em fluxo de duas etapas, e a plataforma confirma a venda em etapa única.

Formato do corpo

Exemplo — `payment.authorized`

```
{
  "id": "b3f1c8de-4a2e-4f0b-9a77-2c5d9e6f1a34",
  "type": "payment.authorized",
  "created_at": "2026-08-14T18:00:00Z",
  "merchant_id": "38457351000107",
  "payment": {
    "id": "0f2b6a51-77c3-4a09-9c1e-7d4b2f8e5a10",
    "status": "authorized",
    "amount": 129.9,
    "currency": "BRL",
    "method": "credit",
    "installments": 3,
    "brand": "visa",
    "card_last4": "1234",
    "payer_name": "Maria Souza",
    "origin": "PAYLINK",
    "payment_link_id": "8c1d0e4b-2f96-4a7d-b3e5-91a0c7d2f846",
    "subscription_id": null,
    "subscription_cycle": null,
    "subscription_external_reference": null,
    "rrn": "123456789012",
    "created_at": "2026-08-14T17:58:12Z"
  },
  "event": "PAYLINK",
  "data": { }
}
```

 Copiar

Exemplo — cobrança de assinatura (recorrência)

```
{
  "id": "047cd0cd-9f31-4c8e-8b2a-51d7e9c4f602",
  "type": "payment.authorized",
  "created_at": "2026-08-17T09:00:04Z",
  "merchant_id": "38457351000107",
  "payment": {
    "id": "70b0d9f7-3c58-4e21-9f0d-6a1b8c2e5d34",
    "status": "authorized",
    "amount": 59.9,
    "currency": "BRL",
    "method": "credit",
    "installments": 1,
    "brand": "visa",
    "card_last4": "1234",
    "payer_name": "Maria Souza",
    "origin": "ONLINE",
    "payment_link_id": null,
    "subscription_id": "3f2a7c10-8e44-4b9a-9c12-7d5e6f8a1b2c",
    "subscription_cycle": 3,
    "subscription_external_reference": "PEDIDO-9987",
    "rrn": "123456789012",
    "created_at": "2026-08-17T09:00:01Z"
  },
  "event": "INTEGRATION",
  "data": { }
}
```

CAMPOS

id string opcional	Identificador único desta entrega. Use para idempotência — descarte entregas repetidas com o mesmo id.
type string opcional	O evento disparado. Um dos valores da tabela acima. Vem nulo no caso raro de a processadora informar um status que ainda não temos mapeado — nesse caso, ignore o evento ou trate pelo campo legado data.
created_at string opcional	Data/hora de geração do evento, em UTC (ISO 8601).
merchant_id string opcional	CNPJ/CPF do estabelecimento ao qual o evento pertence.
payment object opcional	Presente em eventos payment.*. Traz id, status, amount, currency, method, installments, brand, card_last4, payer_name, origin, payment_link_id, subscription_id, subscription_cycle, subscription_external_reference, rrn e created_at do pagamento.
payment.subscription_id string opcional	public_id da assinatura que originou a cobrança — o mesmo id devolvido na criação da assinatura. Null quando a venda não é recorrente (link de pagamento, checkout avulso ou POS).
payment.subscription_cycle number opcional	Número do ciclo cobrado (1 = primeira cobrança). Casa com cycle_number do histórico GET <code>/api/subscriptions/{public_id}/charges</code> . Null fora da recorrência.
payment.subscription_external_reference string opcional	A referência que você enviou em external_reference na criação da assinatura, devolvida sem alteração. Null se você não enviou nenhuma.
payout object opcional	Presente em eventos payout.*. Traz id, status, amount, expected_settlement_date, method e brand da liquidação.

 event e data são campos legados

event e data são mantidos apenas para **compatibilidade com integrações antigas**. data traz o corpo original repassado pela processadora, sem padronização. Para integrações novas, use type e os campos payment/payout.

Cobranças de assinatura

Para marcar a fatura certa como paga, use `payment.subscription_id` — ele traz o `public_id` da assinatura, o mesmo identificador que você guardou ao criá-la — junto de `payment.subscription_cycle`, que diz qual ciclo foi cobrado. Se você preferir trabalhar com o seu próprio identificador, envie `external_reference` na criação da assinatura: ele volta em todo evento desta assinatura em `payment.subscription_external_reference`.

 Não correlacione por nome, valor ou horário

Dois assinantes podem ter o mesmo valor e a cobrança do mês roda em lote — a aproximação erra. `payment.id` identifica a venda, `payment.subscription_id` + `payment.subscription_cycle` identificam a fatura, e o campo `id` do evento serve à idempotência. O mesmo `payment.id` aparece em `payment_id` no histórico de ciclos (GET `/api/subscriptions/{public_id}/charges`), o que fecha a reconciliação também para trás.

Assinatura

Toda requisição de webhook traz estes cabeçalhos:

X-Pagmaxx-Signature string opcional	HMAC-SHA256, em hexadecimal, do corpo exato recebido, calculado com o seu segredo de assinatura.
X-Pagmaxx-Event-Id string opcional	Mesmo valor do campo <code>id</code> do corpo.
X-Pagmaxx-Event-Type string opcional	Mesmo valor do campo <code>type</code> do corpo.

 Calcule o HMAC sobre o corpo bruto

O HMAC precisa ser calculado sobre o **corpo bruto (raw)** da requisição, antes de qualquer parse de JSON. Reserializar o corpo antes de validar a assinatura muda os

bytes e a verificação falha.

Verificação — Node.js

```
import crypto from "crypto";

function isValidSignature(rawBody, signatureHeader, secret) {
  const expected = crypto.createHmac("sha256", secret).update(rawBody).digest("hex");
  const expectedBuf = Buffer.from(expected, "utf8");
  const receivedBuf = Buffer.from(signatureHeader || "", "utf8");

  if (expectedBuf.length !== receivedBuf.length) return false;
  return crypto.timingSafeEqual(expectedBuf, receivedBuf);
}

// use um parser que preserve o corpo bruto (ex.: express.raw)
app.post("/webhooks/pagmaxx", express.raw({ type: "application/json" })), (req, res) => {
  const signature = req.header("X-Pagmaxx-Signature");

  if (!isValidSignature(req.body, signature, process.env.PAGMAXX_WEBHOOK_SECRET)) {
    return res.status(401).end();
  }

  res.status(200).end(); // resposta rápido, processe depois
  const event = JSON.parse(req.body.toString("utf8"));
  // enfileire "event" para processamento assíncrono
});
```

 Copiar

Verificação — Python

```
import hashlib
import hmac

def is_valid_signature(raw_body: bytes, signature_header: str, secret: str) -> bool:
    expected = hmac.new(secret.encode("utf-8"), raw_body, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, signature_header or "")

@app.route("/webhooks/pagmaxx", methods=["POST"])
def pagmaxx_webhook():
    raw_body = request.get_data() # corpo bruto, antes do parse
    signature = request.headers.get("X-Pagmaxx-Signature", "")

    if not is_valid_signature(raw_body, signature, WEBHOOK_SECRET):
        return "", 401

    # resposta rapido e processe de forma assincrona (fila, worker etc.)
    enqueue_webhook_processing(request.get_json())
    return "", 200
```

 Copiar

Reentrega e idempotência

Em **falha de rede** ou resposta **5xx**, a PagMaxx repete a entrega até **5 vezes**, com espera crescente entre as tentativas.

Respostas **4xx** são consideradas definitivas e **não são repetidas** — exceto 408, 425 e 429, que são tratadas como falha temporária e entram no ciclo de reentrega.

 O mesmo evento pode chegar mais de uma vez

Isso é esperado em qualquer sistema de webhooks. Use o campo `id` do corpo (ou o cabeçalho `X-Pagmaxx-Event-Id`) para descartar entregas duplicadas do seu lado.

O que responder

Seu servidor deve responder com qualquer status **2xx** para confirmar o recebimento. O tempo limite de resposta é de **5 segundos**.

 Responda rápido, processe depois

Não faça o processamento pesado antes de responder. Confirme o recebimento assim que validar a assinatura e processe o evento de forma assíncrona (fila, worker, job em background) — se o seu servidor demorar demais para responder, a PagMaxx trata como falha e reentrega o mesmo evento.

[← Gerenciar Assinaturas](#)

[Documentos de Credenciamento →](#)

Documentos de Credenciamento

Durante o credenciamento, o **estabelecimento** e cada **parceiro de split** enviam os documentos de identificação pelo portal. Eles ficam numa fila de conferência no backoffice, onde são baixados e conferidos manualmente — o trâmite de cada documento avança automaticamente conforme a análise do credenciamento avança, sem exigir ação do estabelecimento.

Autenticação

Estes endpoints usam a **sessão do portal** (o mesmo login do estabelecimento ou do backoffice) e **não fazem parte** da API pública de integração por API Key.

Trâmite do documento

Cada documento enviado tem um campo status que representa o trâmite de conferência:

STATUS	SIGNIFICADO
AGUARDANDO_ENVIO	Documento recebido pelo armazenamento e aguardando conferência.
ENVIADO	Documento conferido e considerado válido para o credenciamento.
RECUSADO	Documento recusado; status_note traz o motivo (reenvio necessário).

Nome do tipo de documento

document_type_name no envio e na listagem para o próprio estabelecimento/parceiro pode aparecer com um apelido de exibição mais claro (ex.: "Contrato Social"). Na fila do backoffice, o mesmo campo vem sempre com o **nome oficial** do tipo de documento — é esse nome que quem confere encontra do outro lado.

Documentos do parceiro de split

ENDPOINT	DESCRIÇÃO
GET /api/partners/{partner_id}/documents	Lista os tipos de documento disponíveis e os já enviados para o parceiro.
POST /api/partners/{partner_id}/documents	Envia um novo documento (multipart/form-data).
DELETE /api/partners/{partner_id}/documents/{slug_document}	Remove um documento enviado. slug_document é o id do registro retornado em uploaded[].slug.

CORPO DO ENVIO (MULTIPART/FORM-DATA)

slug_document_type string obrigatório	Identificador do tipo de documento, vindo de types[].slug.
file file obrigatório	PDF, JPG, JPEG ou PNG, até 10 MB.

● 200 — payload de documentos

```
{
  "slug_merchant": "1EB9...C9",
  "kyc_status": "WAITINGDOCUMENTS",
  "kyc_justification": null,
  "term_signed": true,
  "dock_active": false,
  "types": [
    {
      "slug": "2A1E0AAD7D484E3E8628DAF25DFCDE19",
      "name": "Selfie",
      "category": "IDENTIFICATION",
      "required": true,
      "mandatory": true,
      "uploaded_count": 1
    }
  ],
  "uploaded": [
    {
      "slug": "526ba500-0d2e-45c3-ac1d-bceb9aeb0d23",
      "slug_document_type": "2A1E0AAD7D484E3E8628DAF25DFCDE19",
      "name": "Selfie",
      "file_name": "selfie.pdf",
      "file_extension": "pdf",
      "onboarding_status": "AGUARDANDO_ENVIO",
      "dt_insert": "2026-08-20T10:38:00Z",
      "status": "AGUARDANDO_ENVIO",
      "status_note": null
    }
  ]
}
```

 Copiar

STATUS	DETAIL	QUANDO OCORRE
422	Tipo de documento inválido para este parceiro.	slug_document_type não existe para o tipo de pessoa (PF/PJ) do parceiro.
422	Formato inválido. Aceitos: PDF, JPG, PNG.	Extensão do arquivo fora da lista aceita.
422	Arquivo vazio. / Arquivo excede 10 MB.	Arquivo sem conteúdo ou maior que o limite.
409	Documento já encaminhado; não pode mais ser removido.	DELETE em documento cujo status não é AGUARDANDO_ENVIO.
503	Armazenamento de documentos indisponível.	Armazenamento de documentos não configurado ou fora do ar neste ambiente.

Documentos do estabelecimento (EC)

Mesma experiência, para os documentos do próprio estabelecimento (sem parceiro associado).

ENDPOINT	DESCRIÇÃO
GET /api/customer/documents/	Lista os tipos de documento disponíveis e os já enviados pelo estabelecimento.
POST /api/customer/documents/	Envia um novo documento (multipart/form-data).
DELETE /api/customer/documents/{doc_id}	Remove um documento enviado. doc_id é o id do registro retornado em uploaded[].slug.

O corpo do envio é igual ao do parceiro (slug_document_type + file). O corpo de resposta segue o mesmo formato, sem os campos de credenciamento (slug_merchant, kyc_status, kyc_justification, term_signed, dock_active) — só types e uploaded.

STATUS	DETAIL	QUANDO OCORRE
422	Tipo de documento inválido para este estabelecimento.	slug_document_type não existe para o tipo de pessoa (PF/PJ) do estabelecimento.
422	Formato inválido. Aceitos: PDF, JPG, PNG.	Extensão do arquivo fora da lista aceita.
404	Documento não encontrado.	DELETE em doc_id que não pertence ao estabelecimento logado.
409	Documento já encaminhado; não pode mais ser removido.	DELETE em documento cujo status não é AGUARDANDO_ENVIO.
503	Armazenamento de documentos indisponível.	Armazenamento de documentos não configurado ou fora do ar neste ambiente.

Fila de conferência (backoffice)

Endpoints internos do backoffice, restritos a usuários com permissão de conferência de credenciamento.

ENDPOINT	DESCRIÇÃO
GET /internal/admin/onboarding-documents	Fila agrupada por estabelecimento/parceiro, com prazo calculado a partir do documento pendente mais antigo do grupo.
GET /internal/admin/onboarding-documents/{doc_id}/download	Baixa o arquivo original enviado (streaming, com Content-Disposition).
POST /internal/admin/onboarding-documents/{doc_id}/status	Override manual do trâmite.

FILTROS DA FILA (QUERY)

status string opcional	AGUARDANDO_ENVIO ENVIADO RECUSADO. Omitido = todos.
customer_id string (UUID) opcional	Filtra por um estabelecimento específico.
q string opcional	Busca por documento (CPF/CNPJ) ou nome do parceiro.

● 200 — fila agrupada

```
{
  "data": [
    {
      "customer": { "id": "365df920-...", "name": "Customer Teste" },
      "partner": { "id": "57edd69a-...", "name": "SIDE LTDA" },
      "documents": [
        {
          "id": "5f8c2ac0-...",
          "slug_document_type": "...",
          "document_type_name": "Estatutos da Empresa",
          "file_name": "contrato.pdf",
          "content_type": "application/pdf",
          "size_bytes": 204800,
          "status": "AGUARDANDO_ENVIO",
          "status_note": null,
          "created_at": "2026-08-20T11:12:00Z",
          "sent_at": null,
          "rejected_at": null
        }
      ],
      "pending_count": 1,
      "deadline": "2026-08-26T11:12:00Z"
    }
  ],
  "total_count": 1,
  "total_documents": 1
}
```

 Copiar

document_type_name na fila vem sempre com o **nome oficial** do tipo de documento — nunca o apelido de exibição usado na tela do estabelecimento/parceiro.

CORPO DO OVERRIDE (APPLICATION/JSON)

status	ENVIADO ou RECUSADO.
string obrigatório	
note	Motivo. Obrigatório quando status = RECUSADO.
string opcional	

Override — recusar com motivo

```
POST /internal/admin/onboarding-documents/{doc_id}/status
{
  "status": "RECUSADO",
  "note": "Documento ilegível, favor reenviar com foto mais nítida."
}
```

Copiar

STATUS	DETAIL	QUANDO OCORRE
422	status inválido. Use um de: [...]	Filtro status da fila fora de AGUARDANDO_ENVIO/ENVIADO/RECUSADO.
422	status deve ser ENVIADO ou RECUSADO.	Override com status fora dos dois valores aceitos.
422	Motivo obrigatório ao recusar um documento.	Override para RECUSADO sem note preenchido.
404	Documento não encontrado.	doc_id inexistente no download ou no override.
503	Armazenamento de documentos indisponível.	Falha ao ler o arquivo do armazenamento no download.

← Webhooks